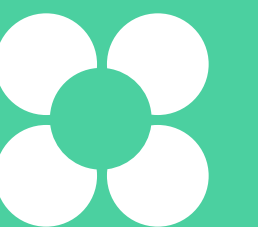


Сегментация и детекция объектов

Егор Конягин

Сотрудник лаборатории высокопроизводительных вычислений и больших данных в Сколтехе



Егор Конягин

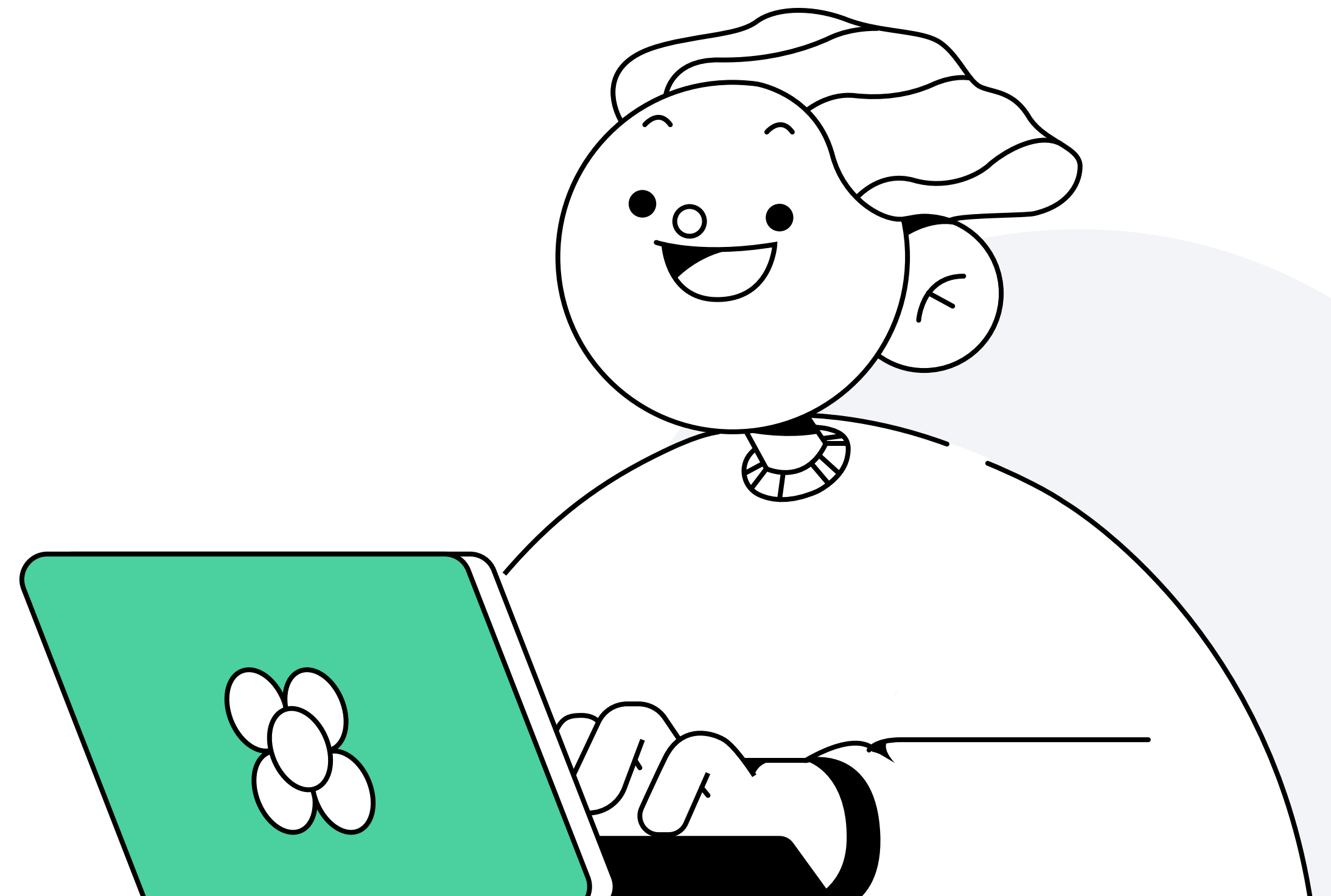
О спикере:

- Сотрудник лаборатории высокопроизводительных вычислений и больших данных в Сколтехе
- Разрабатывал библиотеку Sberbank LightAutoML
- Разрабатывал систему машинного зрения для аэропорта Шереметьево

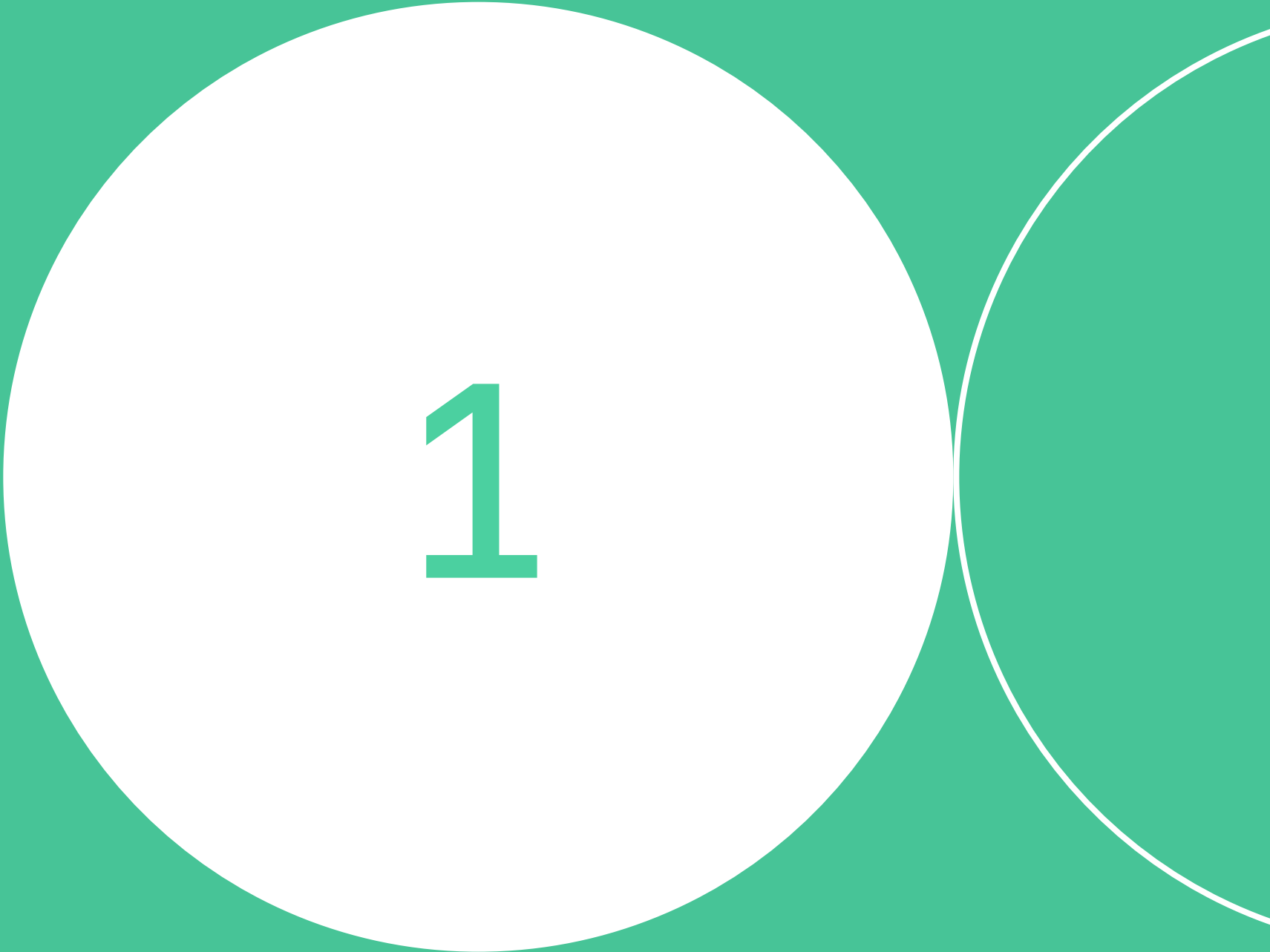


План занятия

- 1 Примеры задач компьютерного зрения
- 2 Сегментация
- 3 Детекция объектов
- 4 Трекинг объектов на видео
- 5 Пример: распознавание жестов



Примеры задач компьютерного зрения

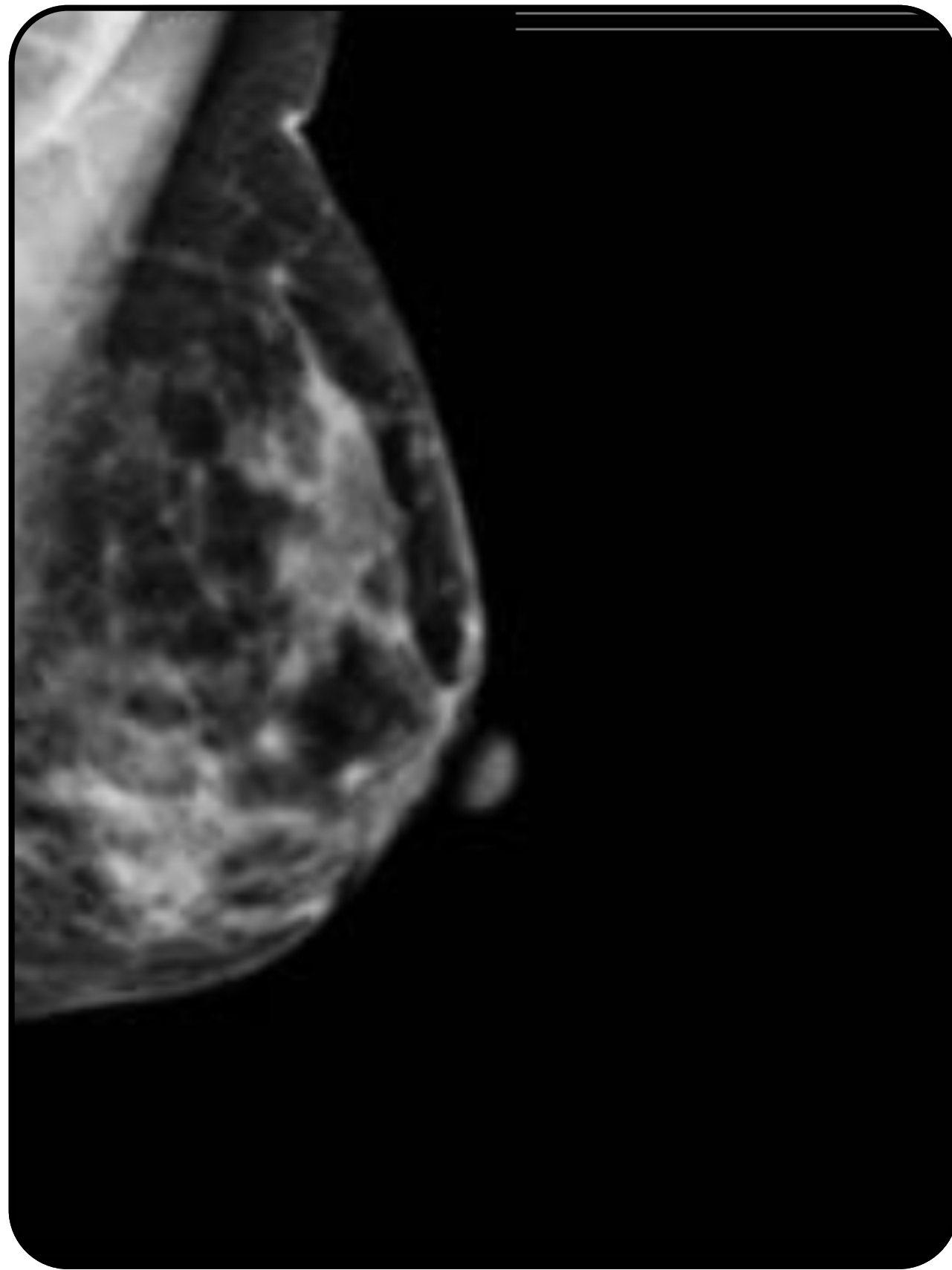


1

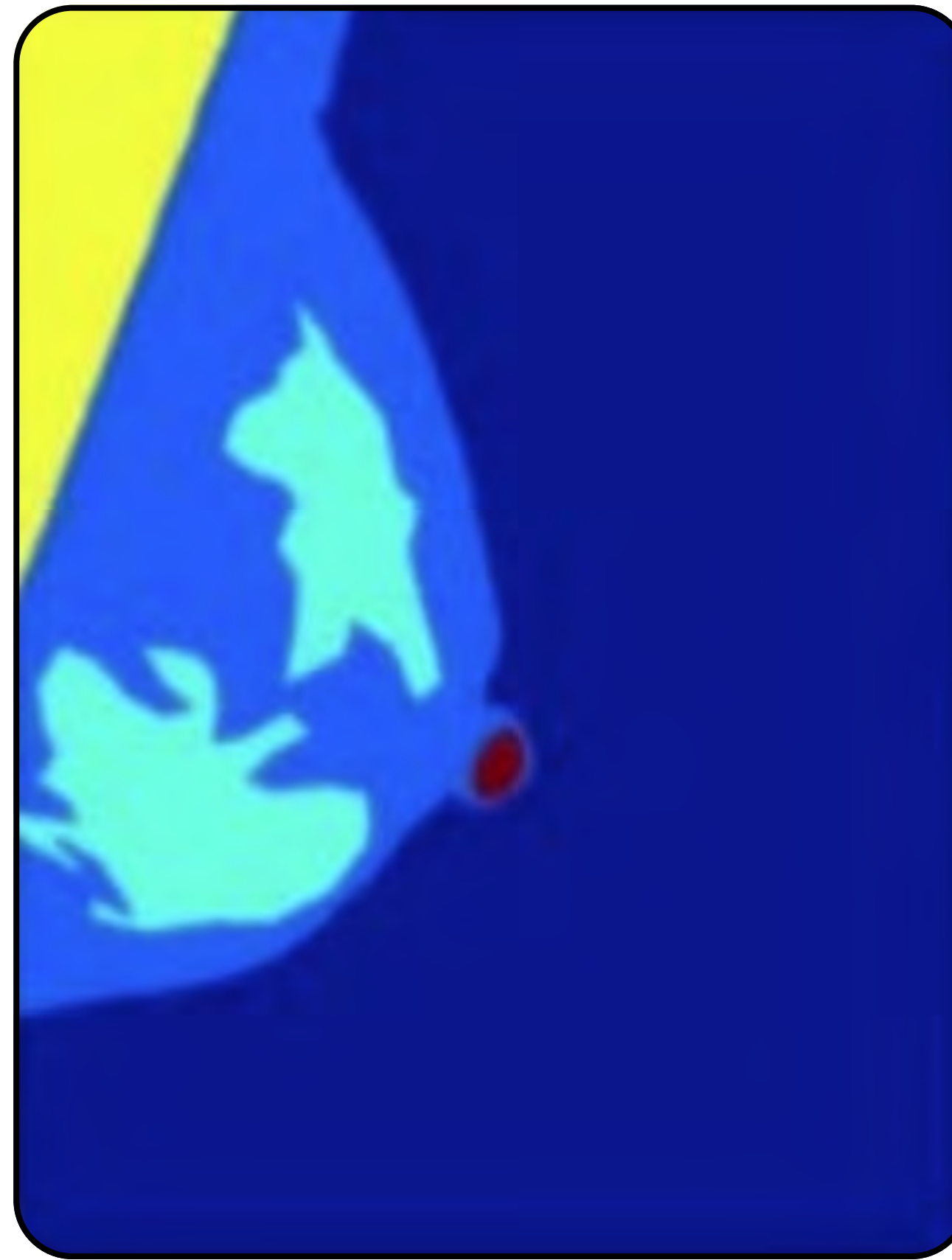
Удаление фона



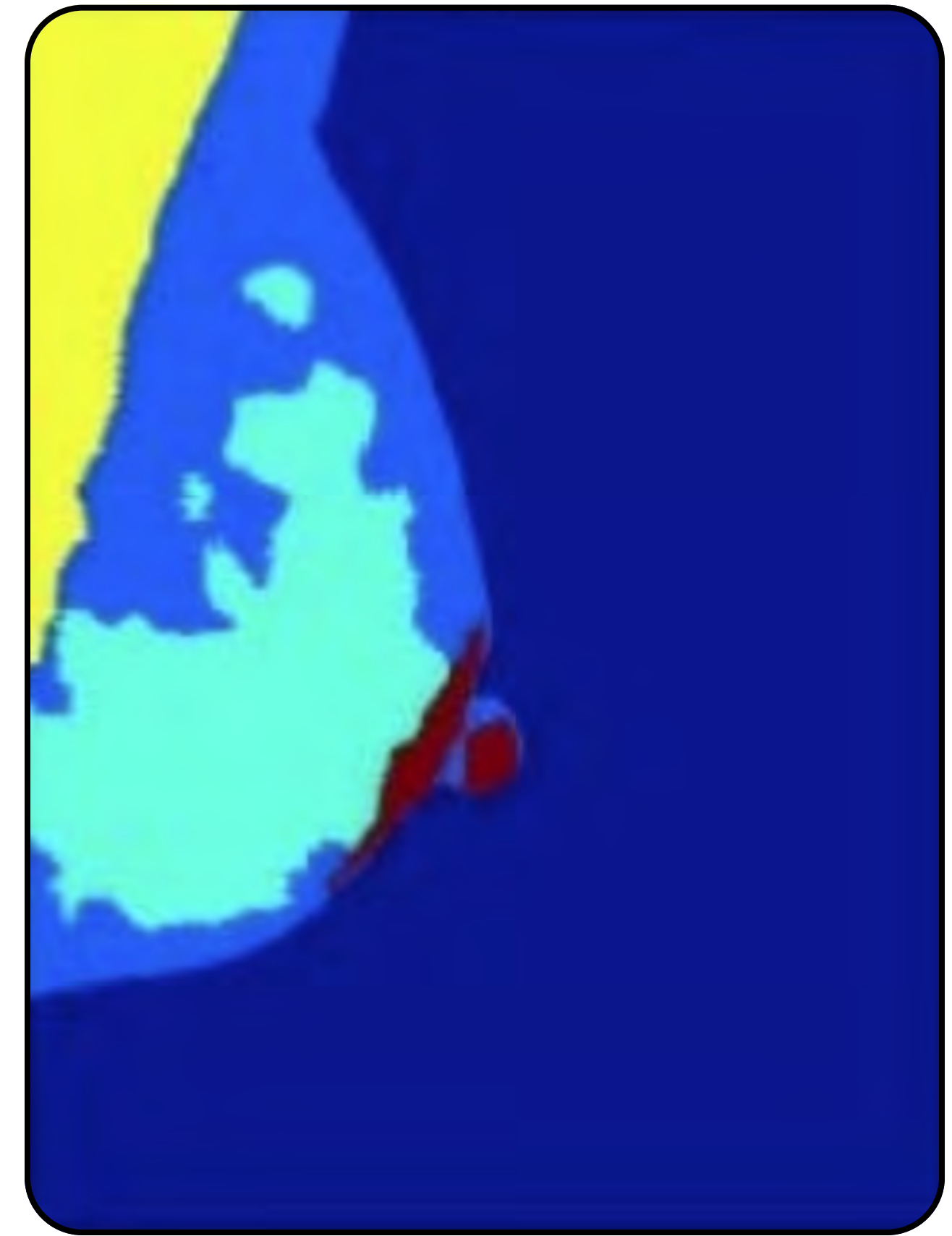
Сегментация медицинских снимков



Оригинал

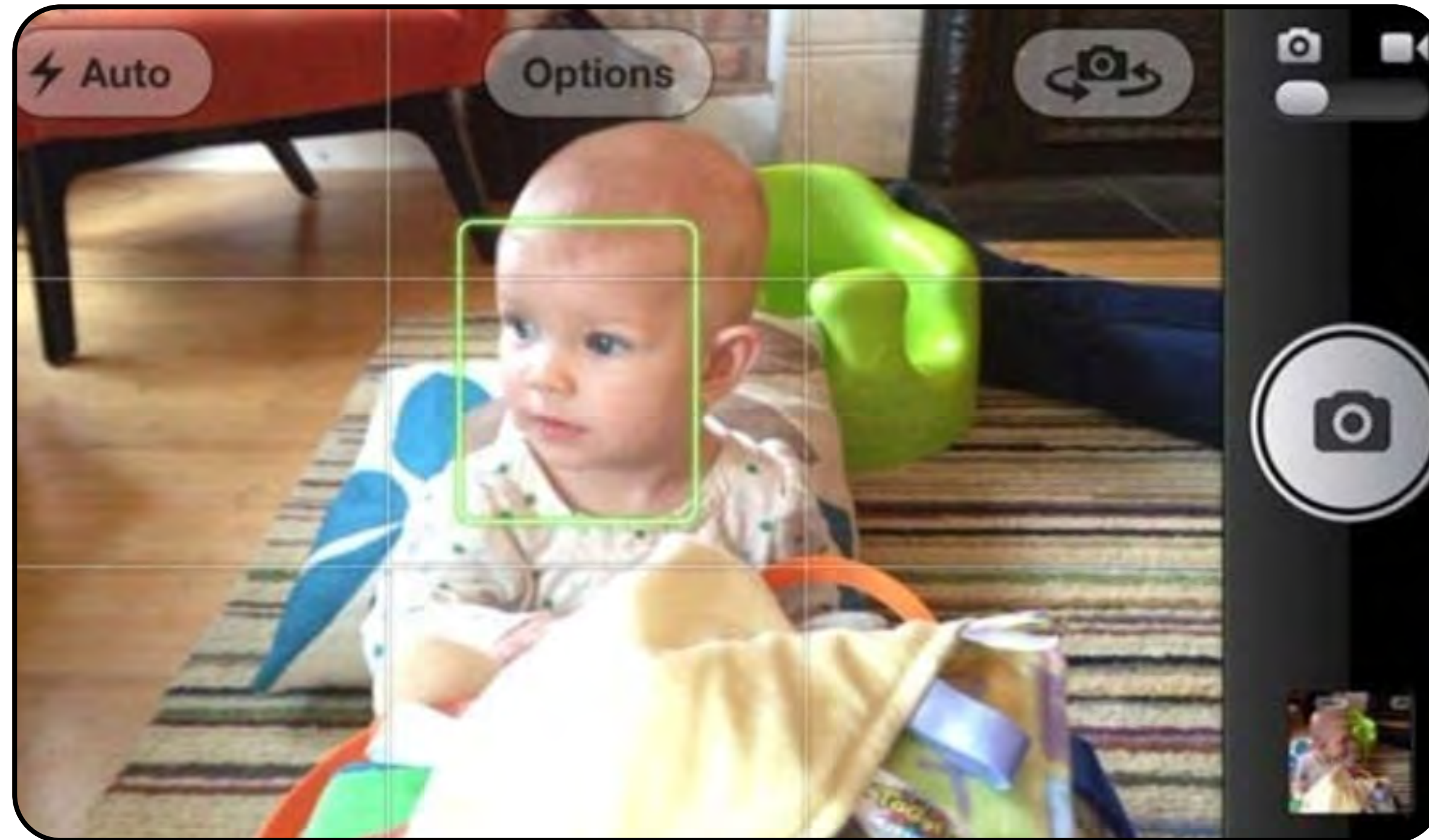


GT

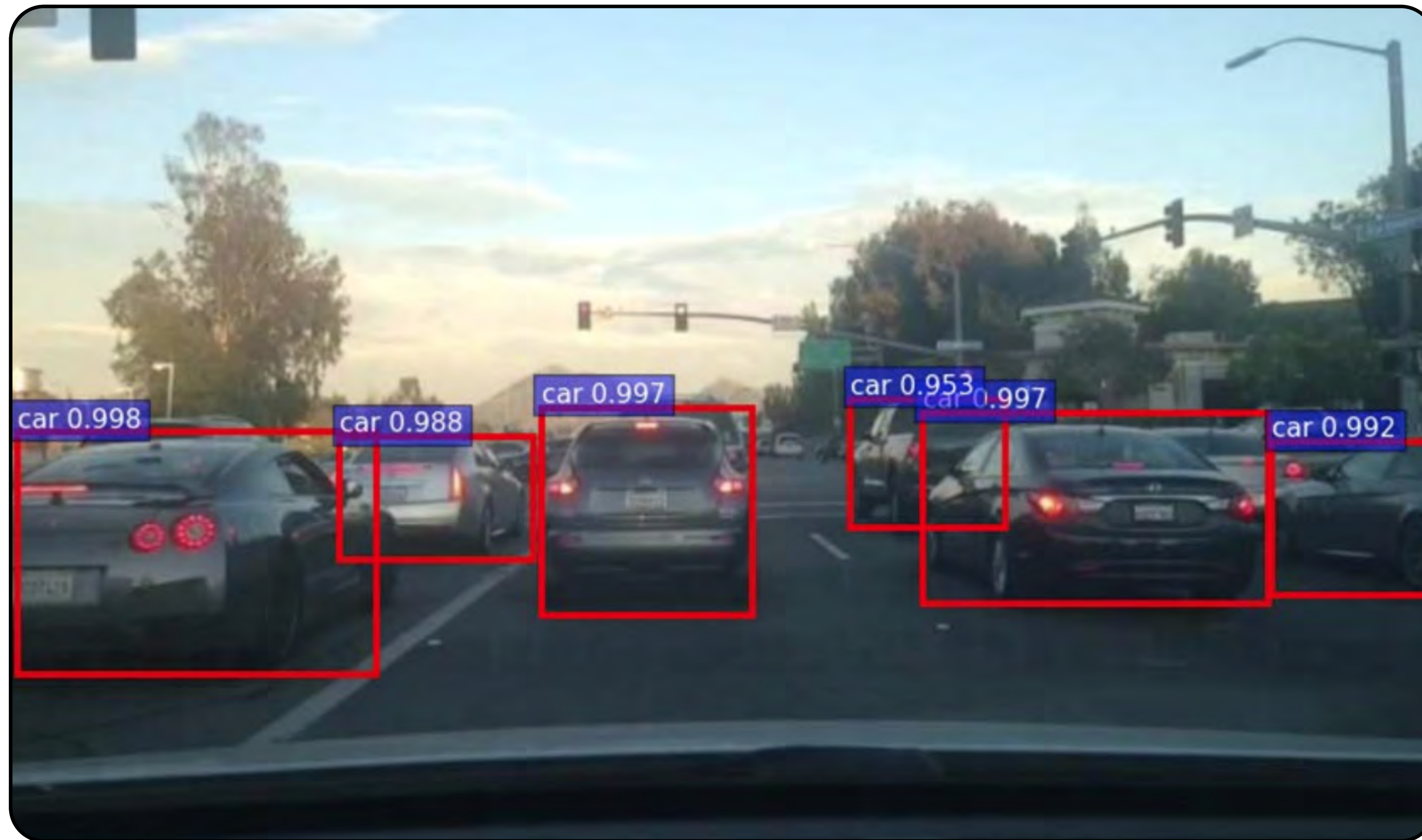


Сегментация на основе DL

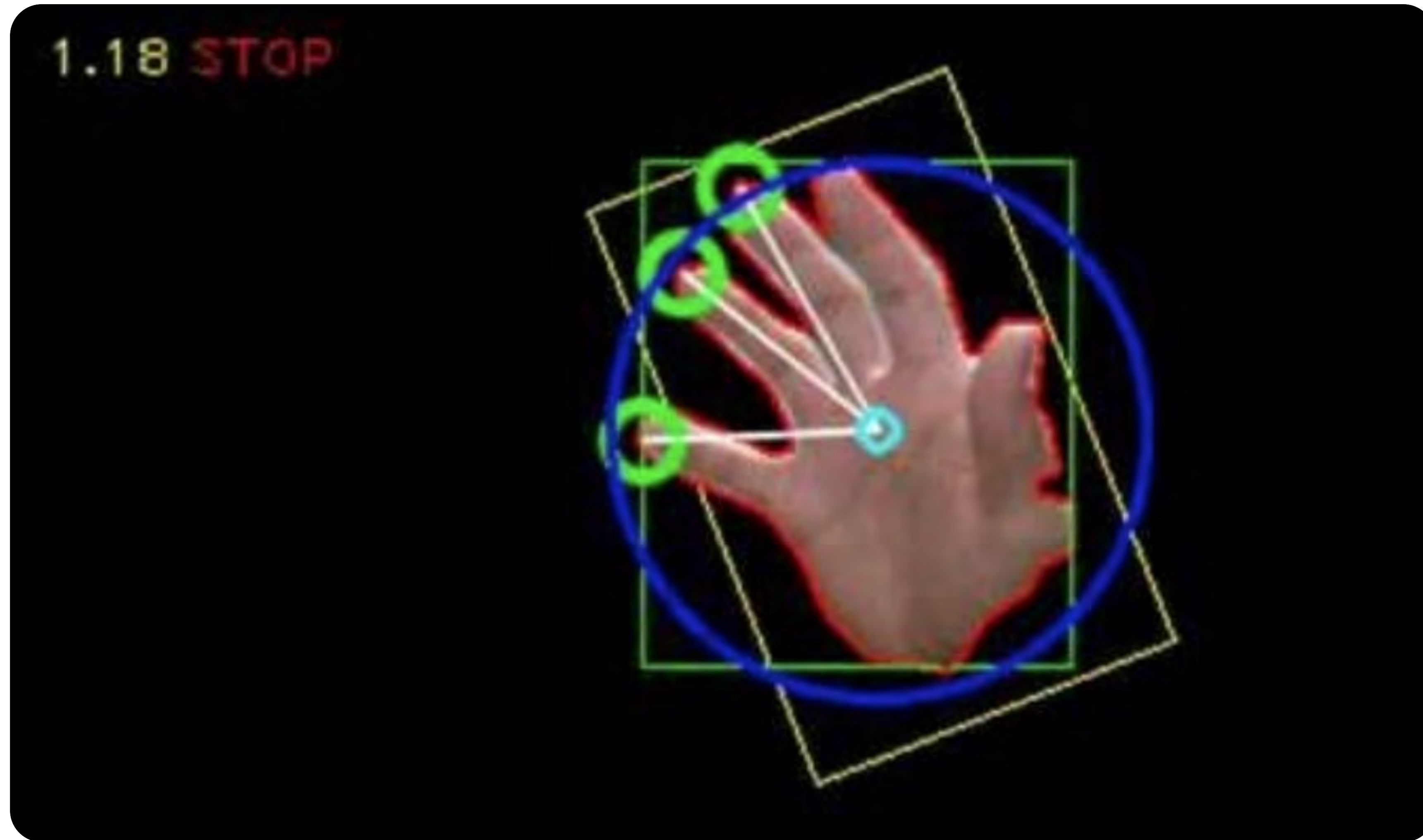
Детекция лица для автофокусировки камеры



Детекция и распознавание объектов для оценки дорожной ситуации



Управление устройством с помощью жестов



Сегментация



2

Основные подходы в сегментации

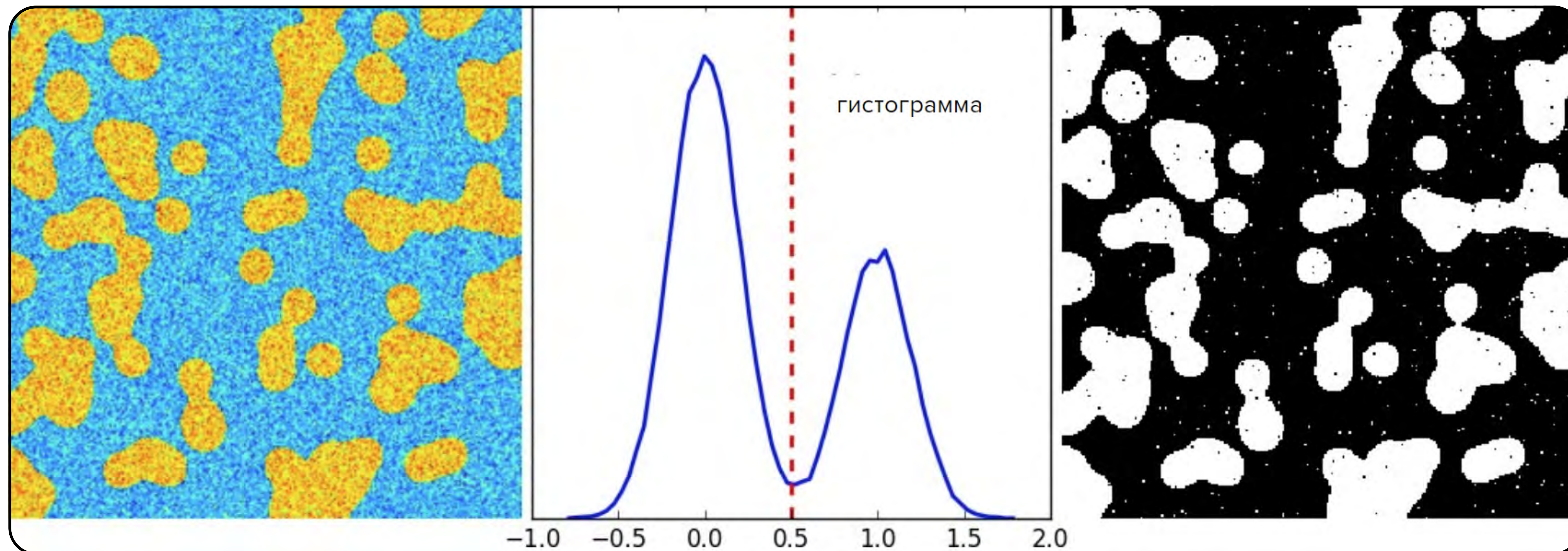
- 1 Сегментация по порогу
- 2 Выделение контура объекта
- 3 Водораздел
- 4 Суперпиксель



Сегментация по порогу



Сегментация по порогу



Сегментация по порогу

`cv2.threshold(src, thresh, maxval, type)`

- `src` — исходное изображение
- `thresh` — пороговое значение
- `maxval` — значение записывается, если интенсивность пикселя выше порога
- `type` — `cv2.THRESH_BINARY`, `cv2.THRESH_TRUNC`, `cv2.THRESH_TOZERO`

Сегментация по порогу

1

**Проста
в реализации**

2

**Неустойчива
к шумам**

3

**Неустойчива
к изменению
освещённости
различных частей
изображения**

Сегментация по порогу

Адаптивный порог

Для каждого пикселя вычисляется усреднённое значение интенсивности в окрестности.

В качестве значения порога используют:

- само среднее значение
- вычисляют взвешенное среднее с гауссовским ядром

Сегментация по пороговым значениям

Адаптивный порог

Исходное
изображение



Глобальное пороговое
значение



Адаптивное определение
среднего порога



Адаптивное определение
порога Гаусса



Сегментация по порогу

`cv2.adaptiveThreshold(src, maxValue, adaptiveMethod, thresholdType, blockSize, C)`

- `src` — исходное одноканальное изображение
- `maxValue` — значение пикселей, для которых интенсивность выше порога
- `adaptiveMethod` — метод оценки порога:
`cv2.ADAPTIVE_THRESH_MEAN_C`,
`cv2.ADAPTIVE_THRESH_GAUSSIAN_C`
- `thresholdType` — тип порога: `cv2.THRESH_BINARY` или `cv2.THRESH_BINARY_INV`
- `blockSize` — размер блока для вычисления порога: 3, 5, 7 и т. д.
- `C` — константа, которая вычитается из оценки значения порога

Адаптивный порог

1

Устойчив
к локальным
изменениям
на изображении

2

Требует больше
вычислительных
ресурсов
по сравнению
с бинарным
порогом

Выделение контура объекта



Сегментация по порогу

1

Находит границы
объекта
на изображении

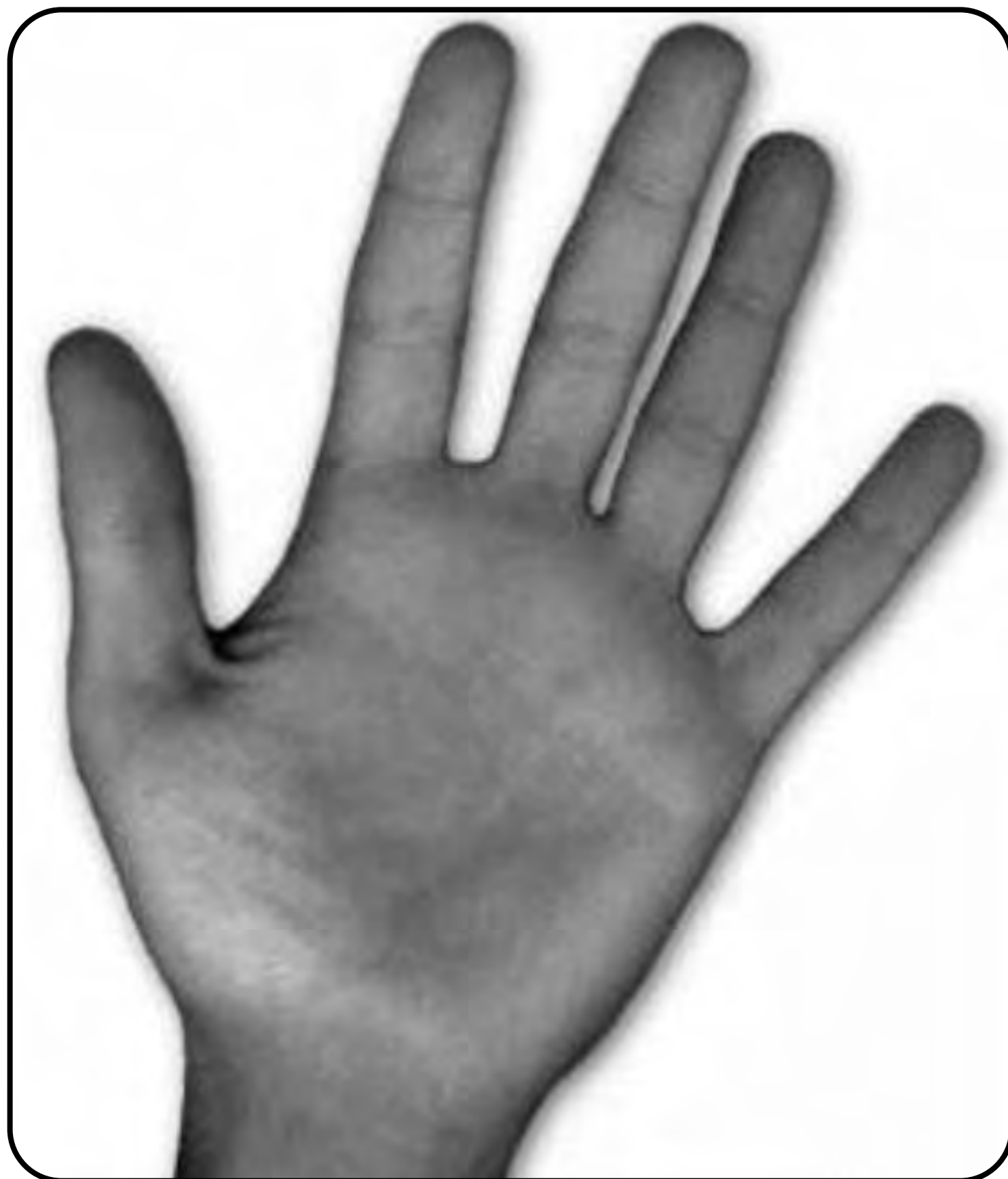
2

Добавляет
информацию
о форме объекта

3

Можно использовать
для детекции
и распознавания
объектов

Выделение контура объекта



Выделение контура объекта

Для поиска контура необходимо бинаризовать исходное изображение.

Бинаризовать можно:

- по порогу
- применить предобработку с помощью детектора граней

Чтобы получить информацию о форме объекта, полученные контуры необходимо аппроксимировать полигоном

Выделение контура объекта

`cv2.findContours(image, mode, method[, contours[, hierarchy[, offset]])` → `contours, hierarchy`

- `image` — бинаризованное изображение
- `mode` определяет, какие контуры необходимо найти:
`cv2.CV_RETR_EXTERNAL, cv2.CV_RETR_TREE`
- `method` — метод поиска и фильтрации точек контура
- `contours` — массив контуров, каждый контур — это массив точек
- `hierarchy` задаёт иерархию вложенности контуров
- `offset` задаёт сдвиг точек контура

Выделение контура объекта

1

Метод требует
предобработки
бинаризации
изображения

2

Время работы
зависит
от сложности
входного
изображения

3

Позволяет
получить
информацию
о форме сегмента

Водораздел (watershed)



Водораздел

- 1 Изображение представляется в виде рельефа
- 2 Точки с высокой интенсивностью — возвышенности
- 3 Точки с низкой интенсивностью — низины
- 4 Помимо интенсивности, используют фактор расстояния от пикселя до грани

Начинаем заполнять низины

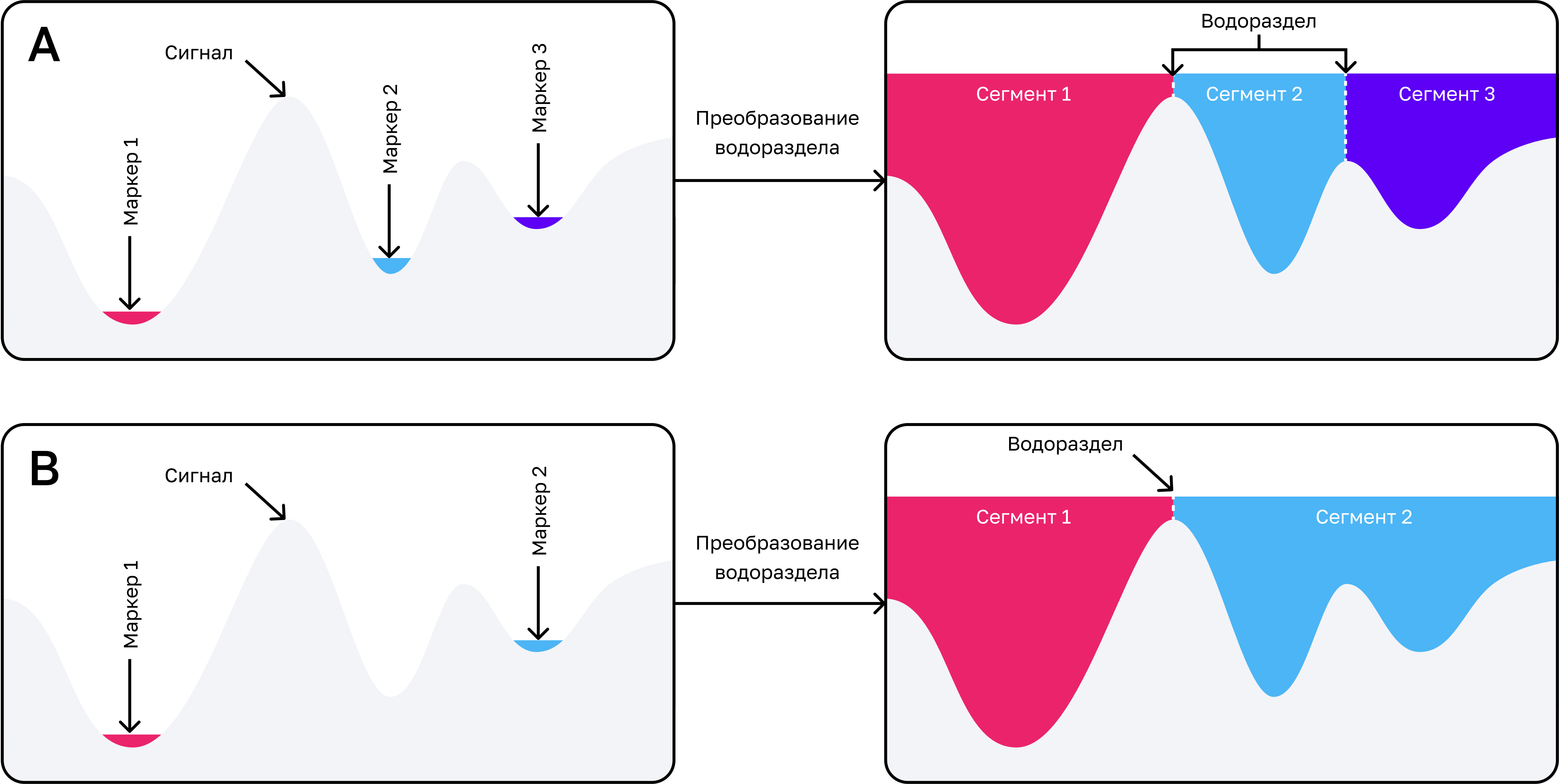
1

Там, где
сталкиваются
области из разных
бассейнов,
проходит граница
объекта

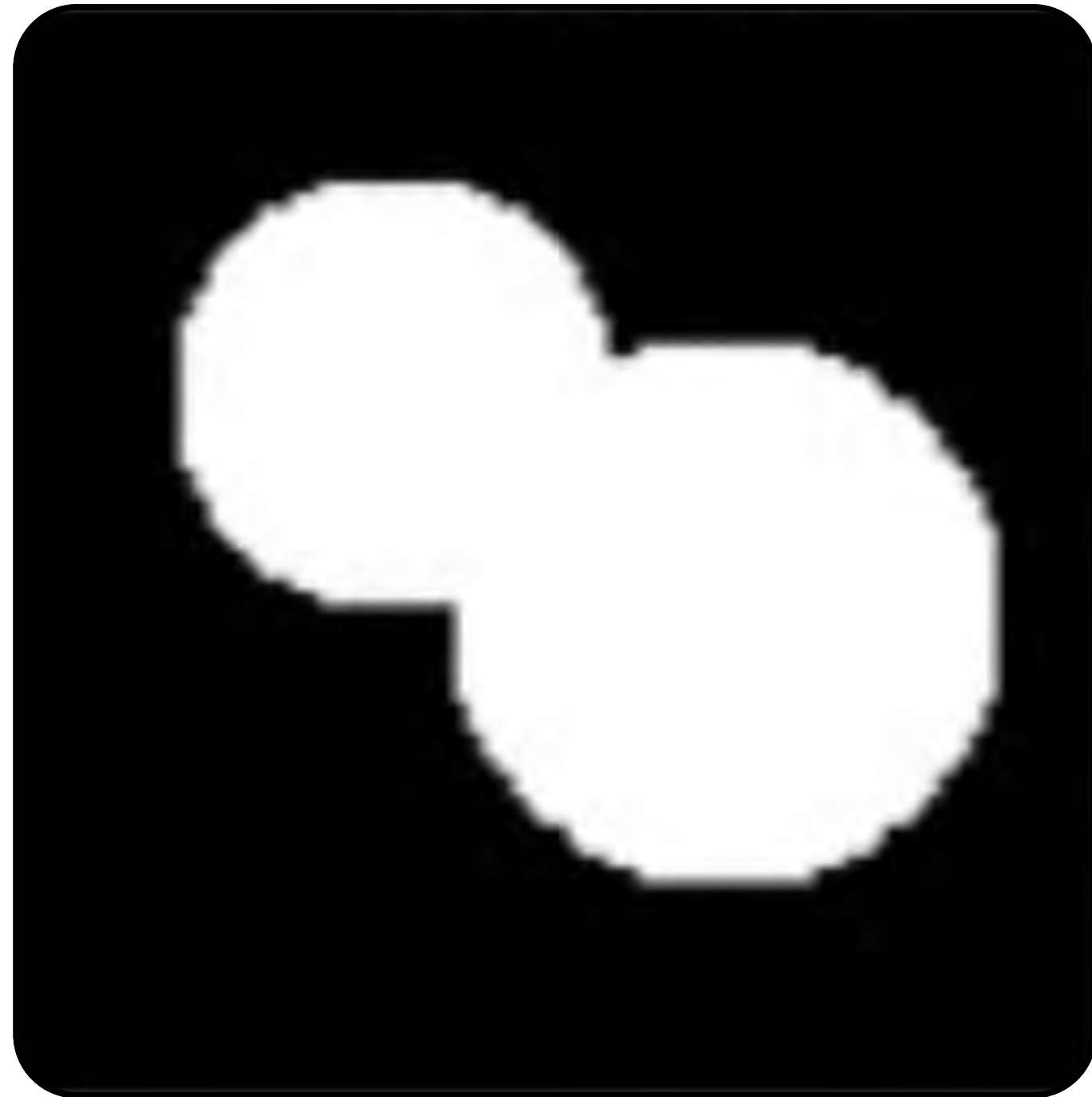
2

Заполняем до тех
пор, пока все
вершины
не скроются
под водой

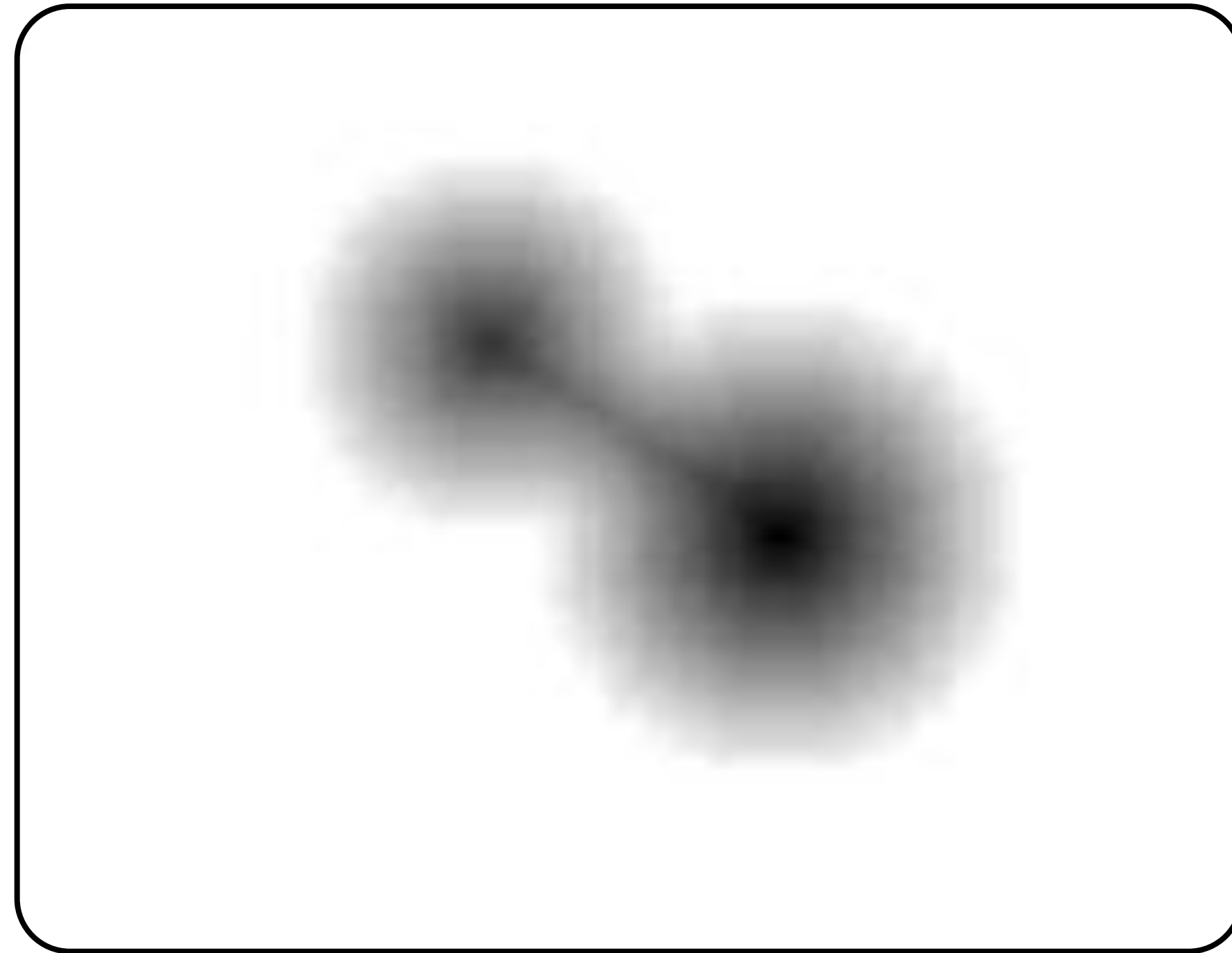
Метод водораздела



Метод водораздела



Перекрывающиеся объекты



Расстояния



Раздельные объекты

Метод водораздела

1

Из-за шума
и неидеальной
структуры
сегменты могут
объединяться
в один

2

Необходимо
указать начальное
приближение –
маркеры
для сегментации

Суперпиксель (super pixel)



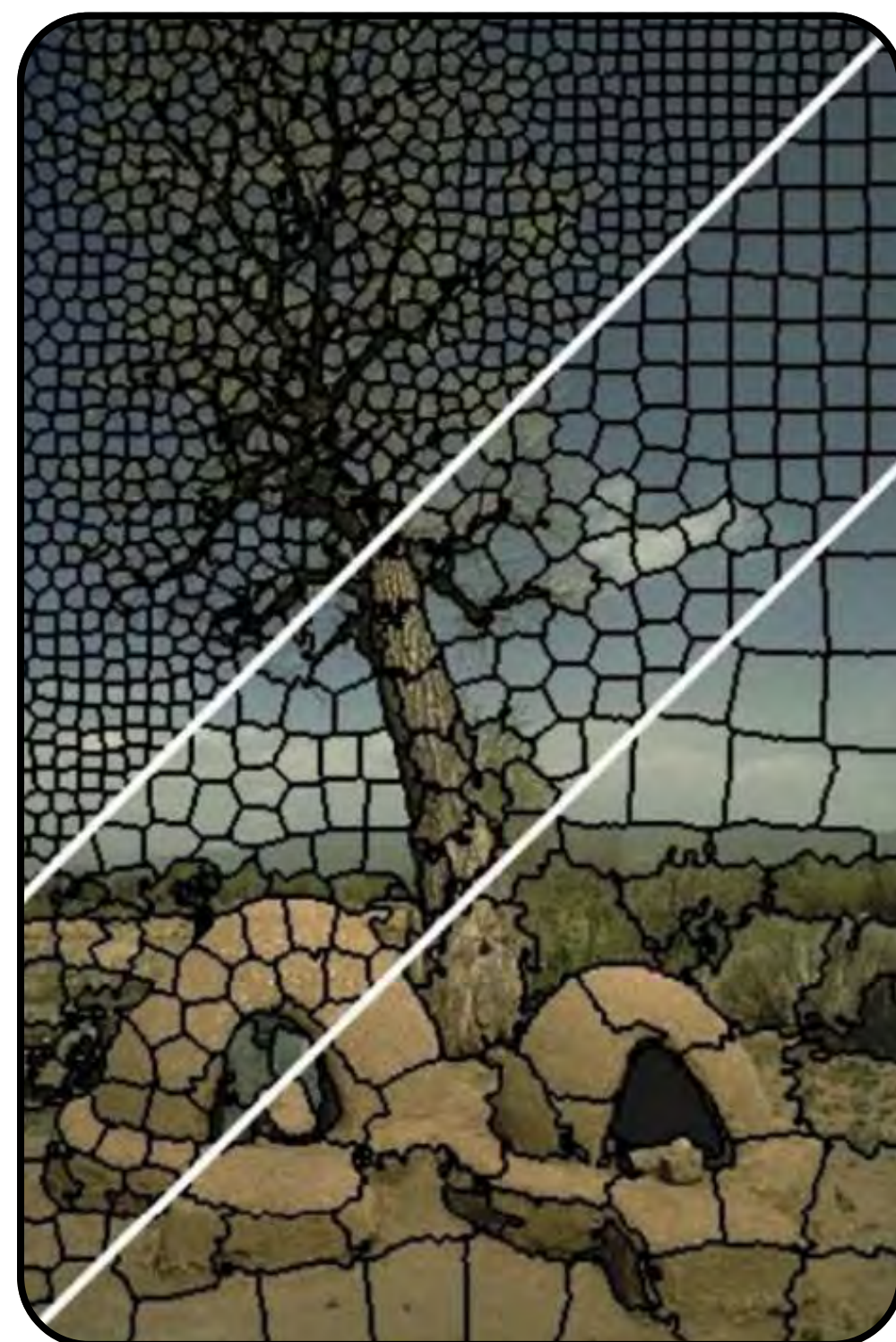
Суперпиксель

Объединение нескольких соседних пикселей на изображении в единую область.

- Объединение происходит по признакам: цвет, текстура
- В результате объединения пикселей получается упрощённое представление изображения. Его можно использовать в задачах распознавания

Суперпиксель

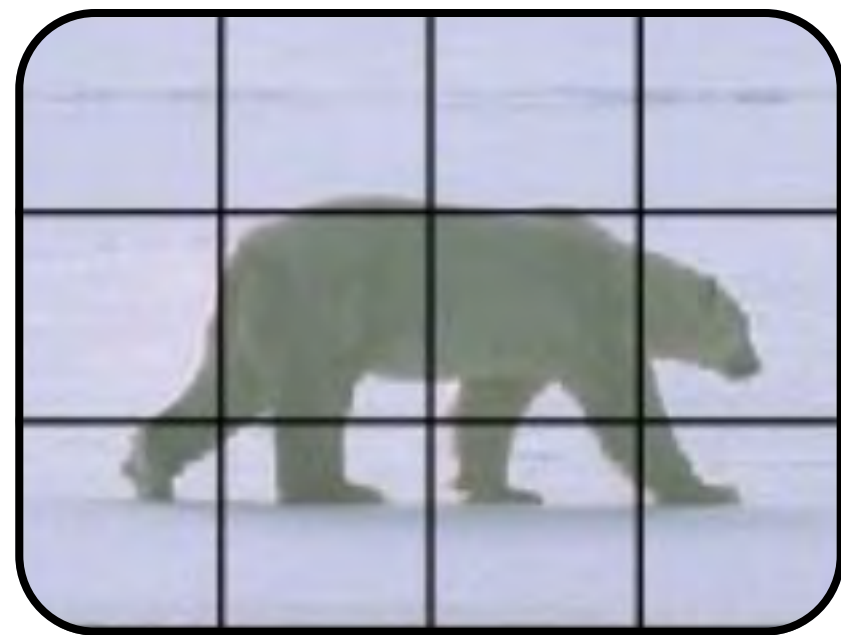
SLIC с простой линейной итеративной кластеризацией:



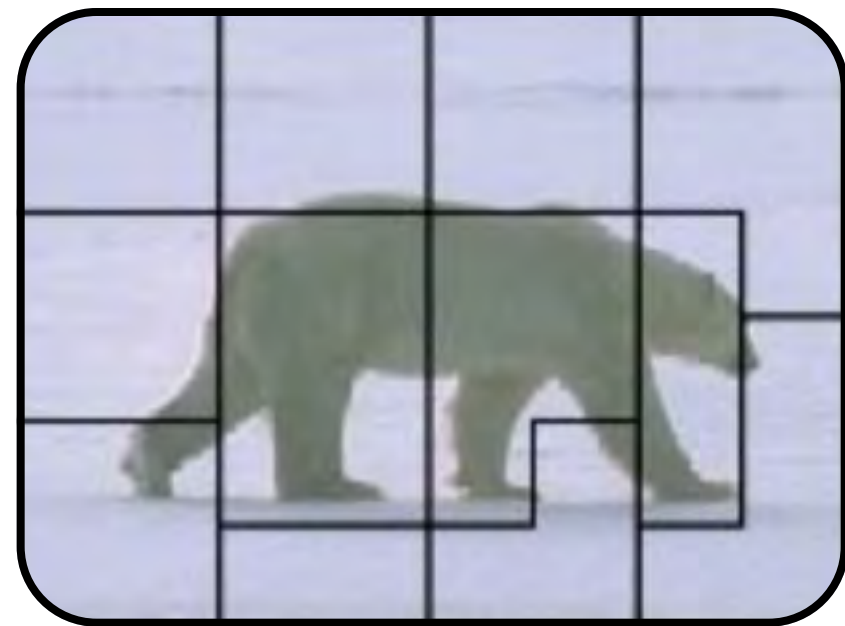
Изображение сегментировано с использованием алгоритма на суперпиксели приблизительного размера 64, 256 и 1024 пикселей.
Суперпиксели компактны, однородны по размеру и хорошо прилегают к границам области

Суперпиксель

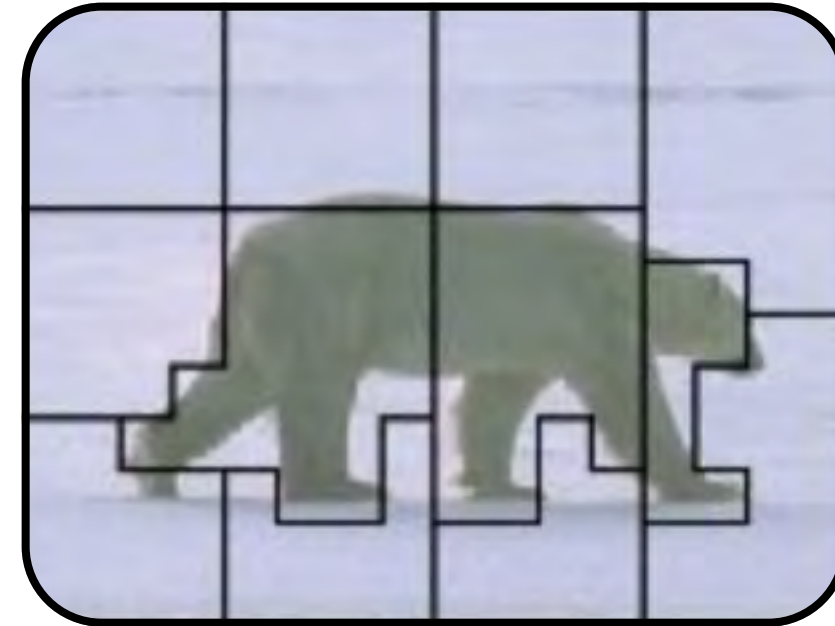
Superpixels Extracted via Energy-Driven Sampling: суперпиксели, извлечённые с помощью выборки, управляемой энергией



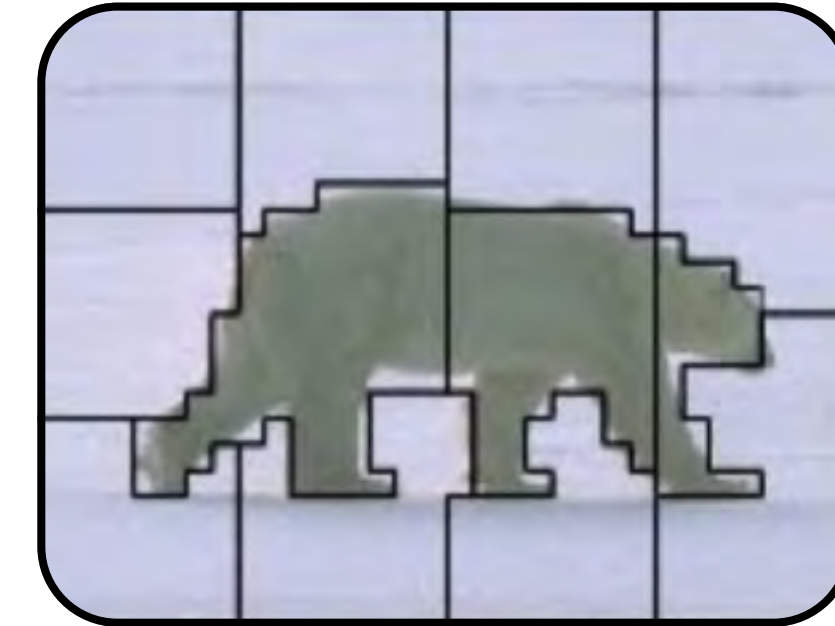
Инициализация



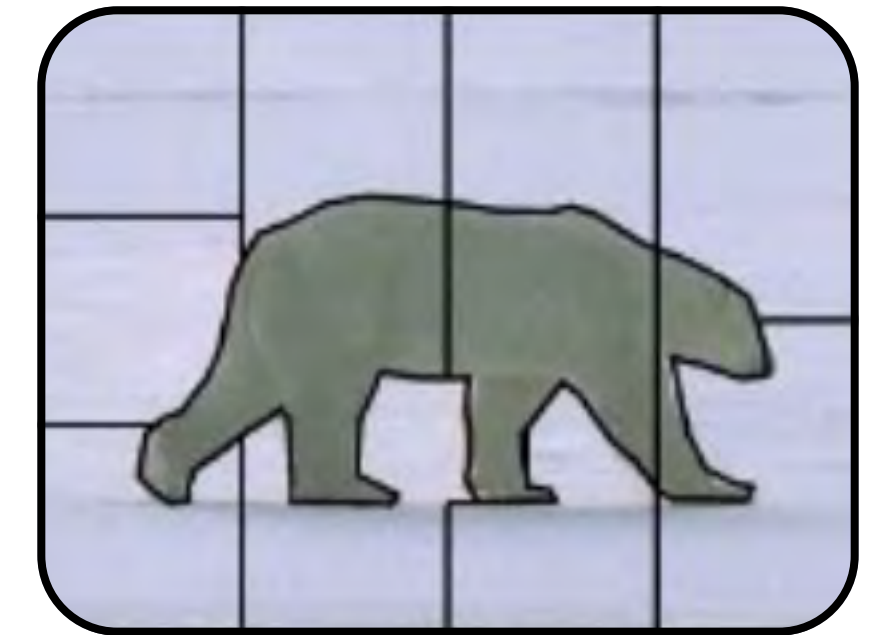
Самое большое
обновление
блока



Среднее
обновление
блока



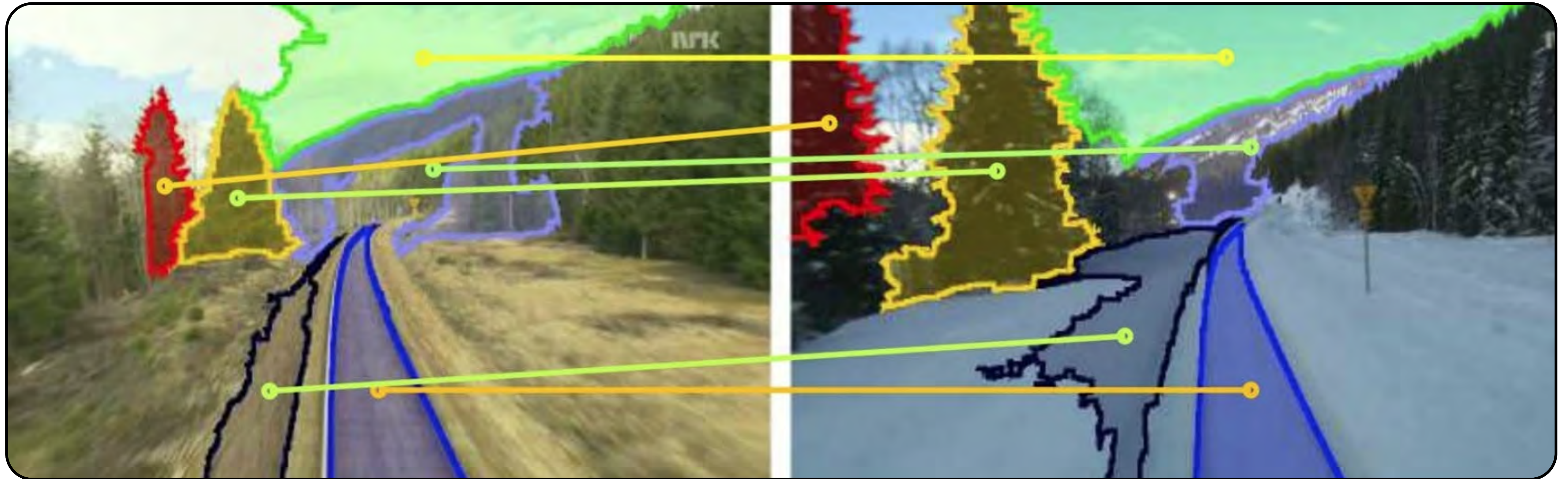
Наименьшее
обновление
блока



Обновление
на уровне
пикселей

Суперпиксель

Распознавание географических мест на фото при изменении окружения



Детекция обьектов

Детектор лиц на изображения



3

Постановка задачи

Детекция:

- поиск объекта на изображении
- в результате получаем область на изображении с соответствующим объектом
- в некоторых случаях выдаётся оценка вероятности детекции

Детектор лиц на изображениях



Детектор лиц на изображении

Детектор по деталям

- Отдельно распознаются части лица: глаза, нос, рот
- Части объединяются в результат детекции

Детектор лица в целом

- Сканирующее окно проходит по изображению
- Каждая часть изображения оценивается: похожа ли она на лицо?
- Есть ограничение на допустимый размер окна

Подготовка данных

1

Подготавливаем
положительные
и отрицательные
примеры
изображений

2

Как правило,
положительных
примеров больше,
чем
отрицательных

3

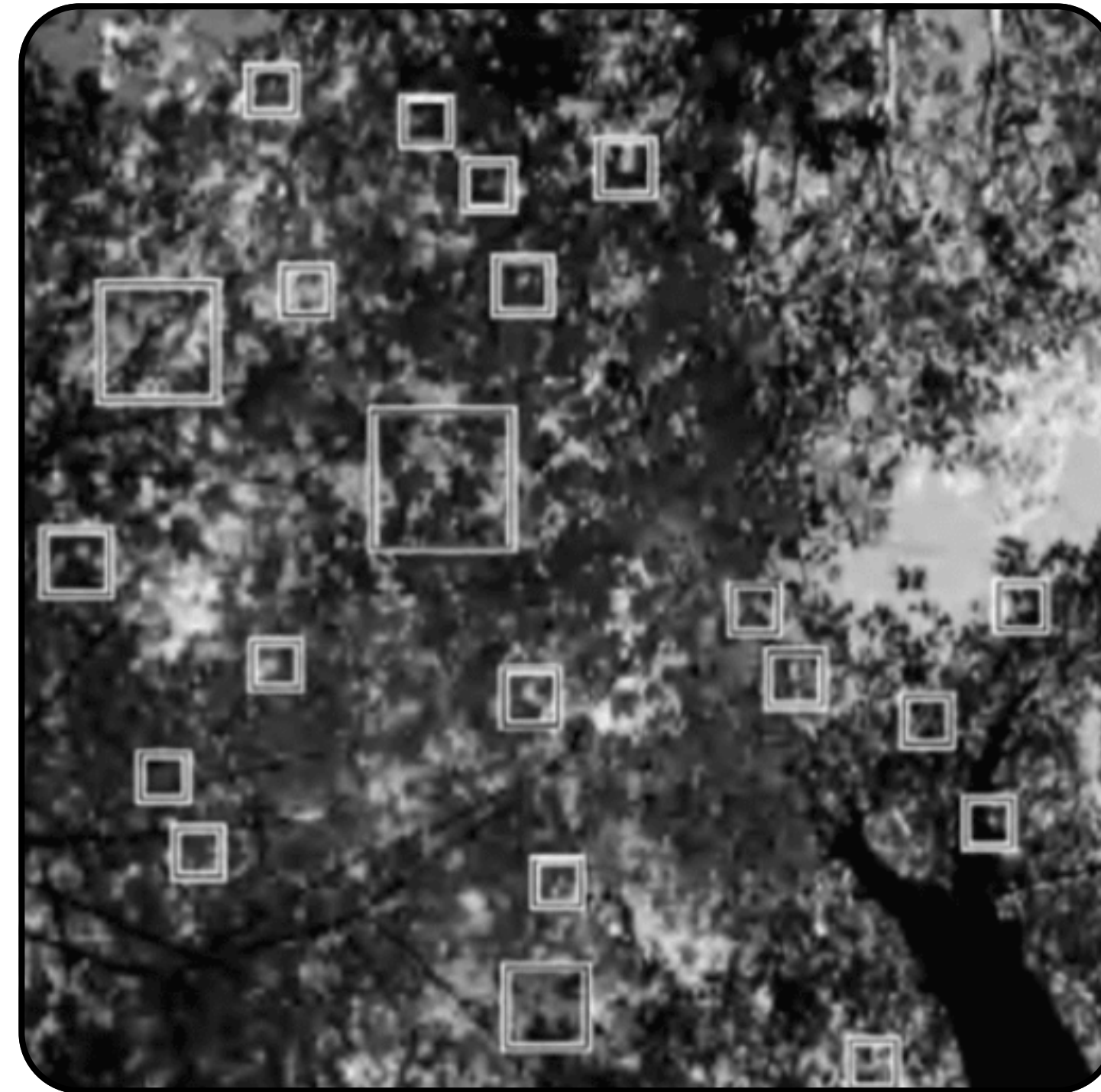
Для баланса
выборки можно
применить
аугментацию
изображений

Детектор лиц на изображении

Аугментация изображения:

- масштабирование
- поворот
- сдвиг
- зеркальное отображение
- вычитание градиента, эмуляция освещённости
- выравнивание гистограммы интенсивности

Подготовка данных



Обучение модели

1

**РСА +
кластеризация**

2

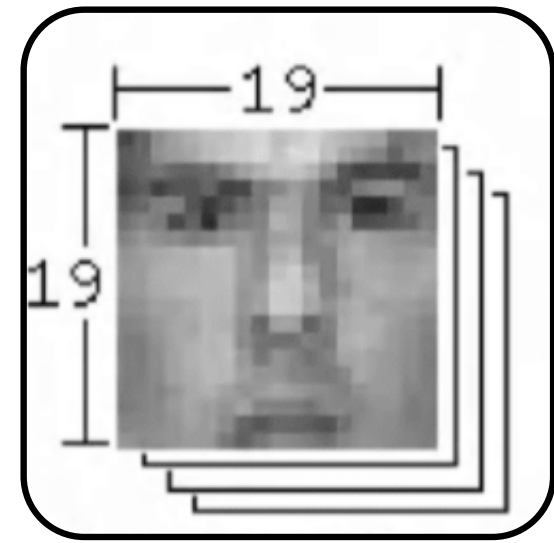
**Бустинг
Viola-Jones**

Кластеризация + PCA

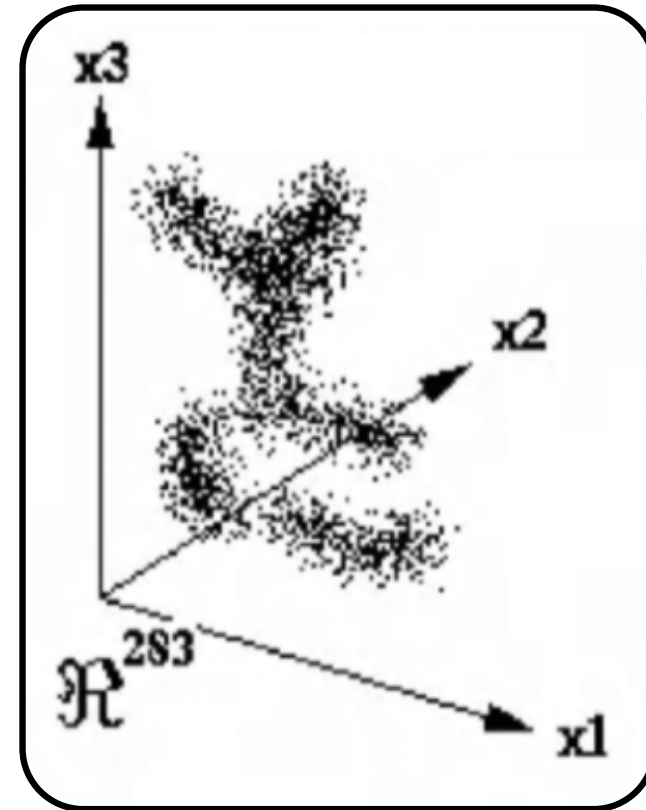
- Снижаем размерность изображения с помощью PCA-преобразования
- Кластеризуем изображения в каждой группе на 6 кластеров
- В результате получается 12 центроидов
- Расстояние от изображения до центроида используем в качестве фактора для обучения
- Обучаем модель: SVM, Random forest, Neural network

[Probabilistic visual learning for object detection](#)

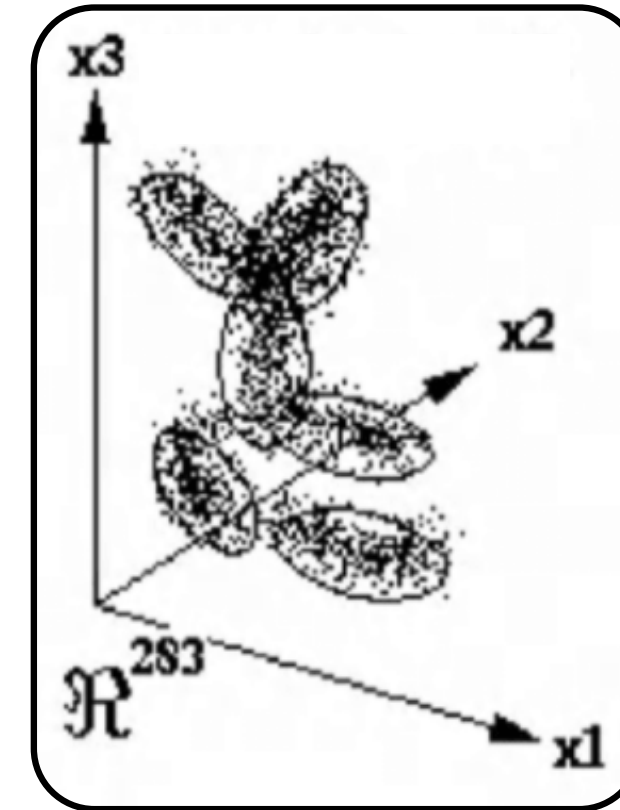
Кластеризация + PCA



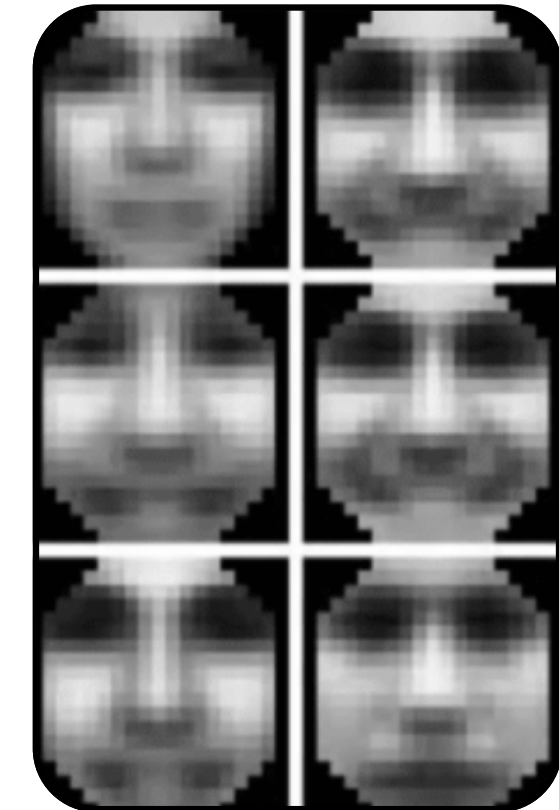
Образцы для построения
векторного
представления лиц



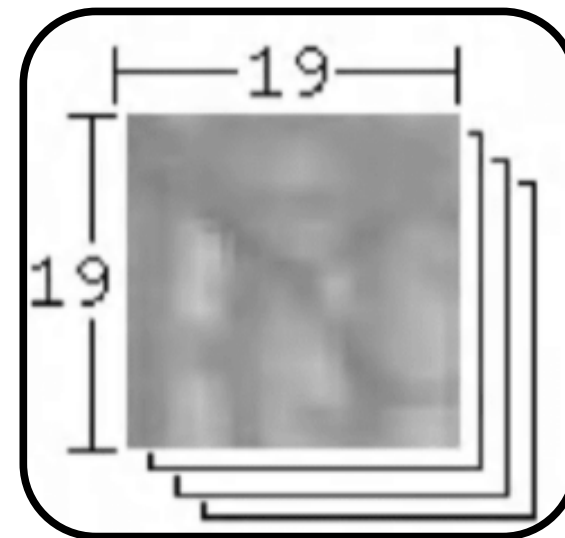
Распределение
образцов с лицами



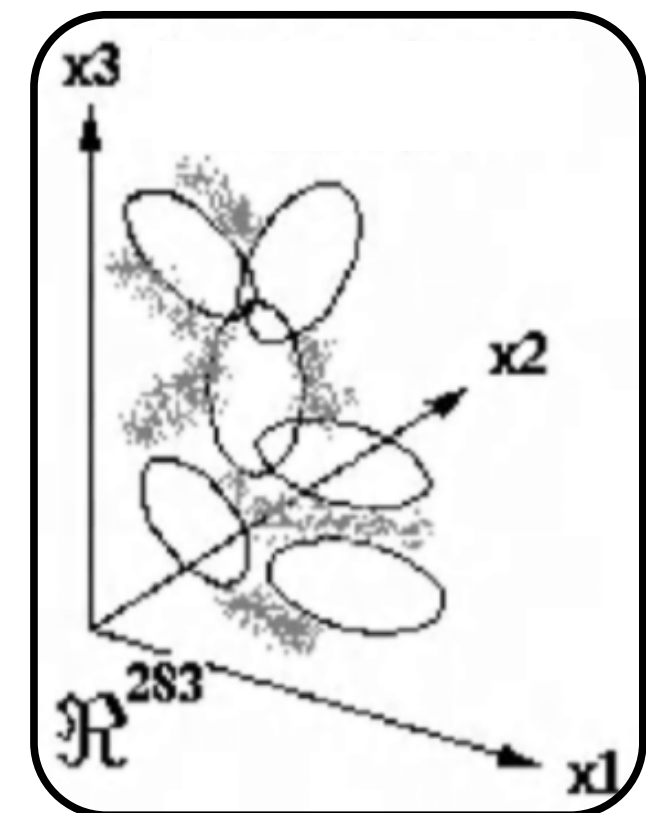
Аппроксимация
гауссовыми кластерами



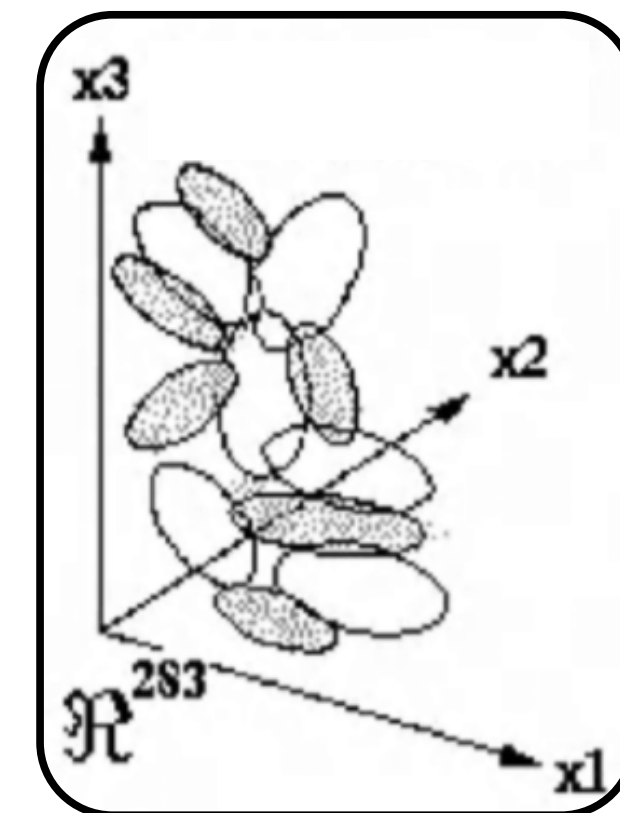
Центроиды
лица



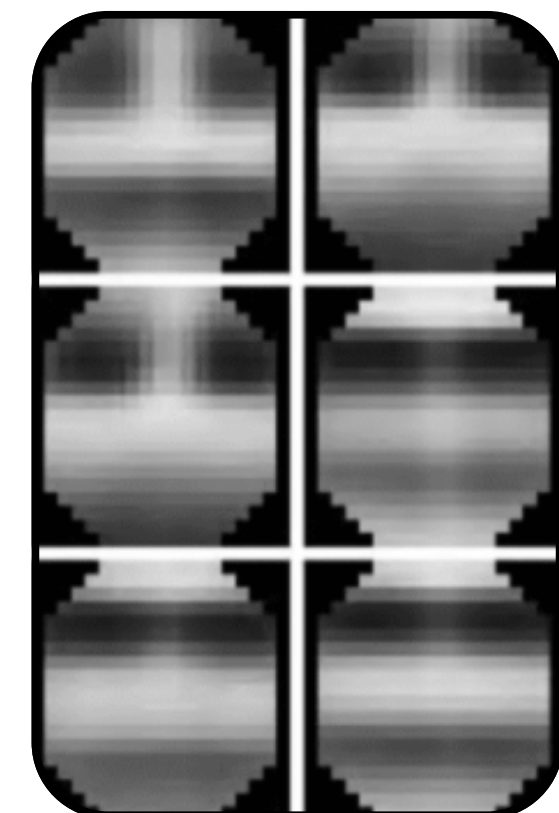
Образцы, не содержащие
лица, для лучшей
настройки векторных
представлений лиц



Распределение
образцов без лиц



Аппроксимация
гауссовыми кластерами



Центроиды
без лица

Бустинг Viola-Jones

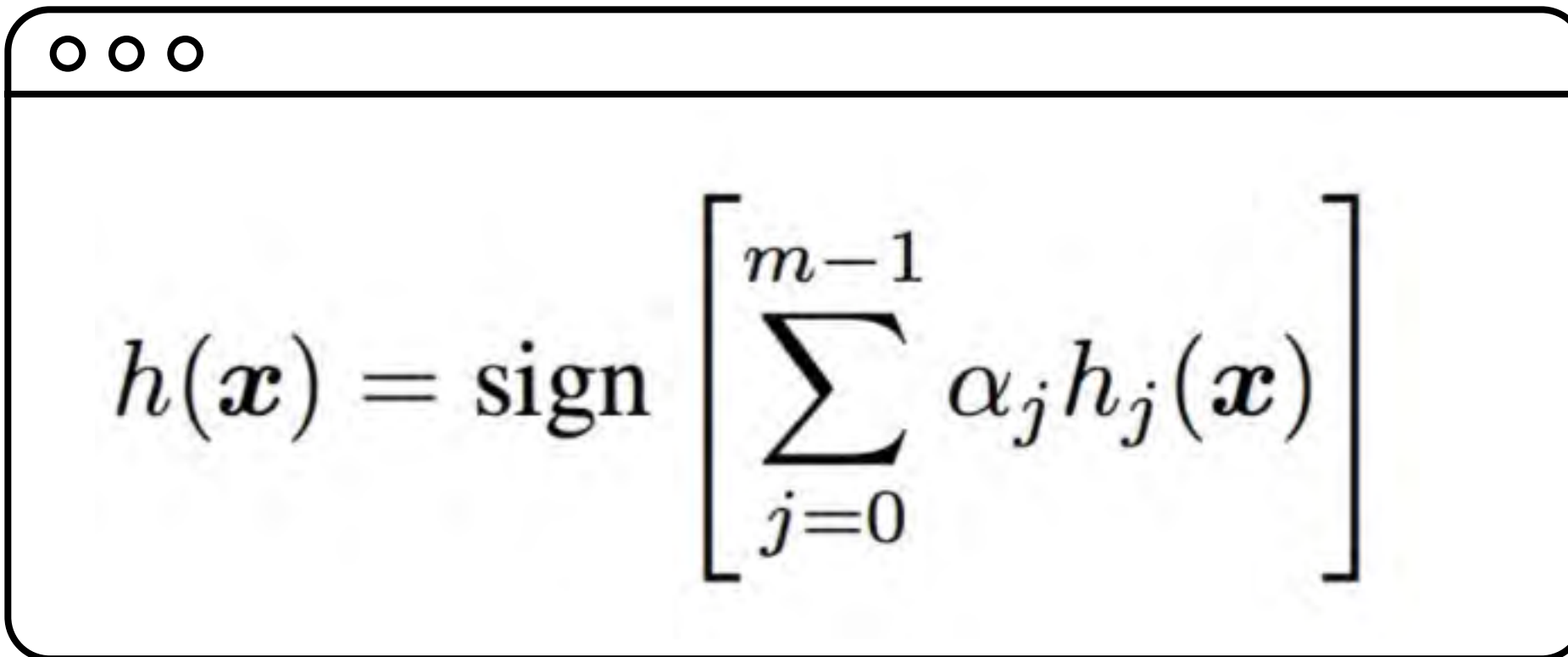
1

Детектор Viola-Jones

2

**Подход
с использованием
бустинга в задачах
компьютерного
зрения**

Бустинг Viola-Jones


$$h(\mathbf{x}) = \text{sign} \left[\sum_{j=0}^{m-1} \alpha_j h_j(\mathbf{x}) \right]$$

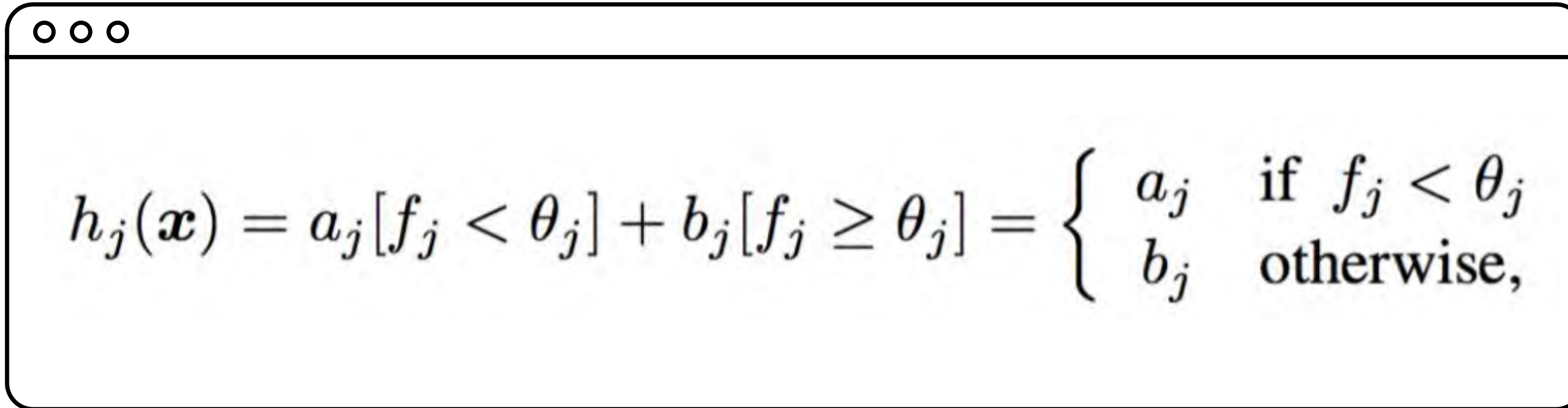
$h(\mathbf{x})$ — финальный классификатор

m — число классификаторов в ансамбле

$h_j(\mathbf{x})$ — базовый классификатор

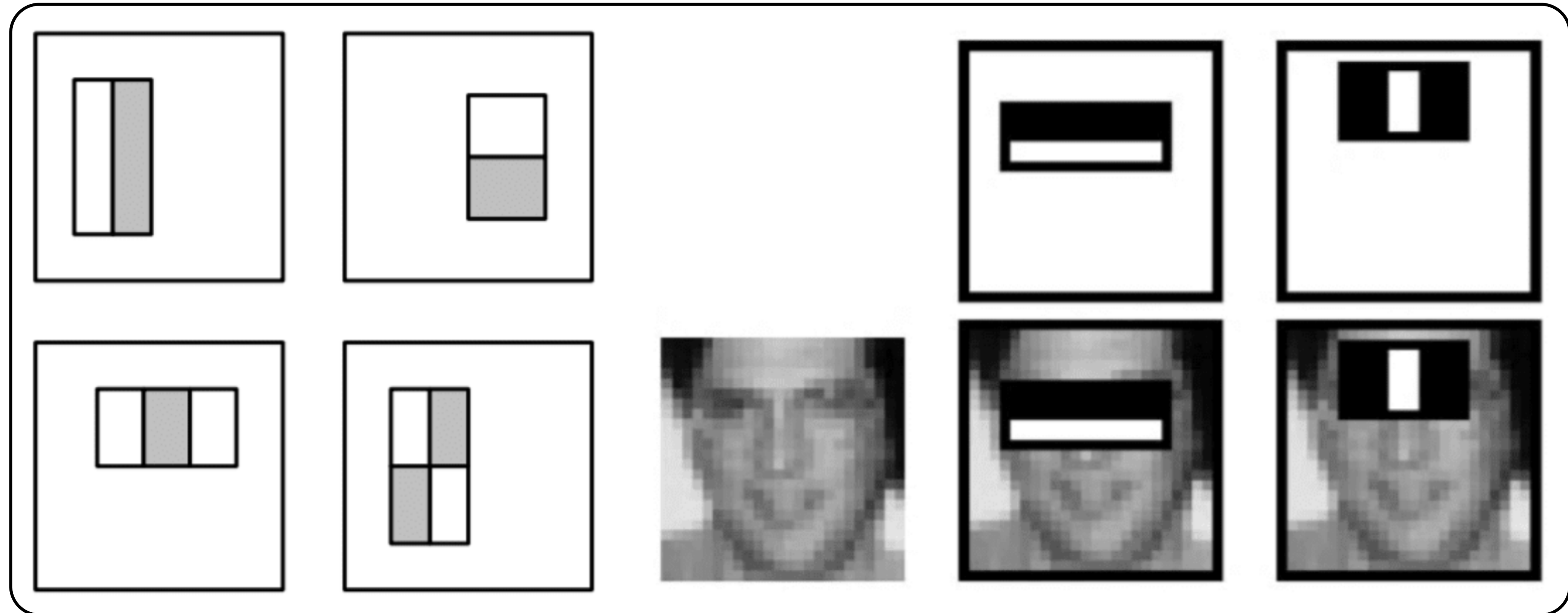
α_j — вес базового классификатора (зависит от ошибки)

Бустинг Viola-Jones


$$h_j(\mathbf{x}) = a_j[f_j < \theta_j] + b_j[f_j \geq \theta_j] = \begin{cases} a_j & \text{if } f_j < \theta_j \\ b_j & \text{otherwise,} \end{cases}$$

- **a_j** — решение принимается в случае, если значение фактора f меньше порога θ
- **b_j** — принимается в противном случае

Бустинг Viola-Jones



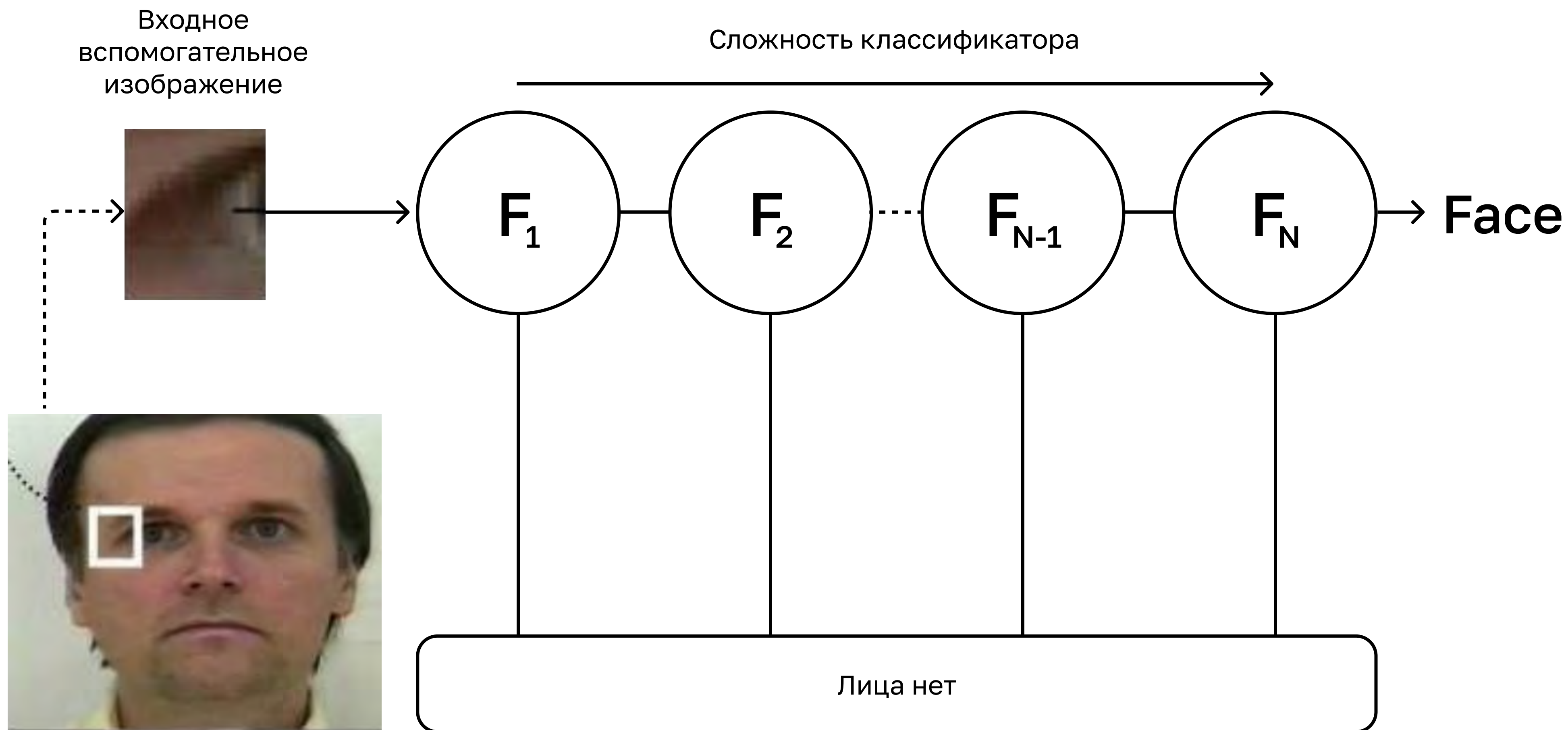
Бустинг Viola-Jones

- При инициализации присваиваем всем объектам в выборке одинаковый вес
- На очередном шаге j среди множества простых классификаторов выбираем тот, который даёт наибольший прирост качества
- Вычисляем коэффициент α , который пропорционален приросту качества с учётом весов объектов
- Увеличиваем вес объектов, которые классифицированы неверно

Бустинг Viola-Jones

- Ориентировочное число простых моделей в ансамбле – 3 000
- Поиск скользящим окном по изображению большим ансамблем может занимать слишком много времени
- На практике строят каскады ансамблей меньшего размера
- Область классифицируется очередным ансамблем в каскаде, только если предыдущий ансамбль классифицировал область как положительную

Бустинг Viola-Jones



Бустинг Viola-Jones

1

Предобученные
модели
для opencv

2

OpenCV
предоставляет
утилиты, чтобы
подготовить
данные для
обучения
детектора

Бустинг Viola-Jones

1

Алгоритм
сравнительно
быстро вычисляет
детекции

2

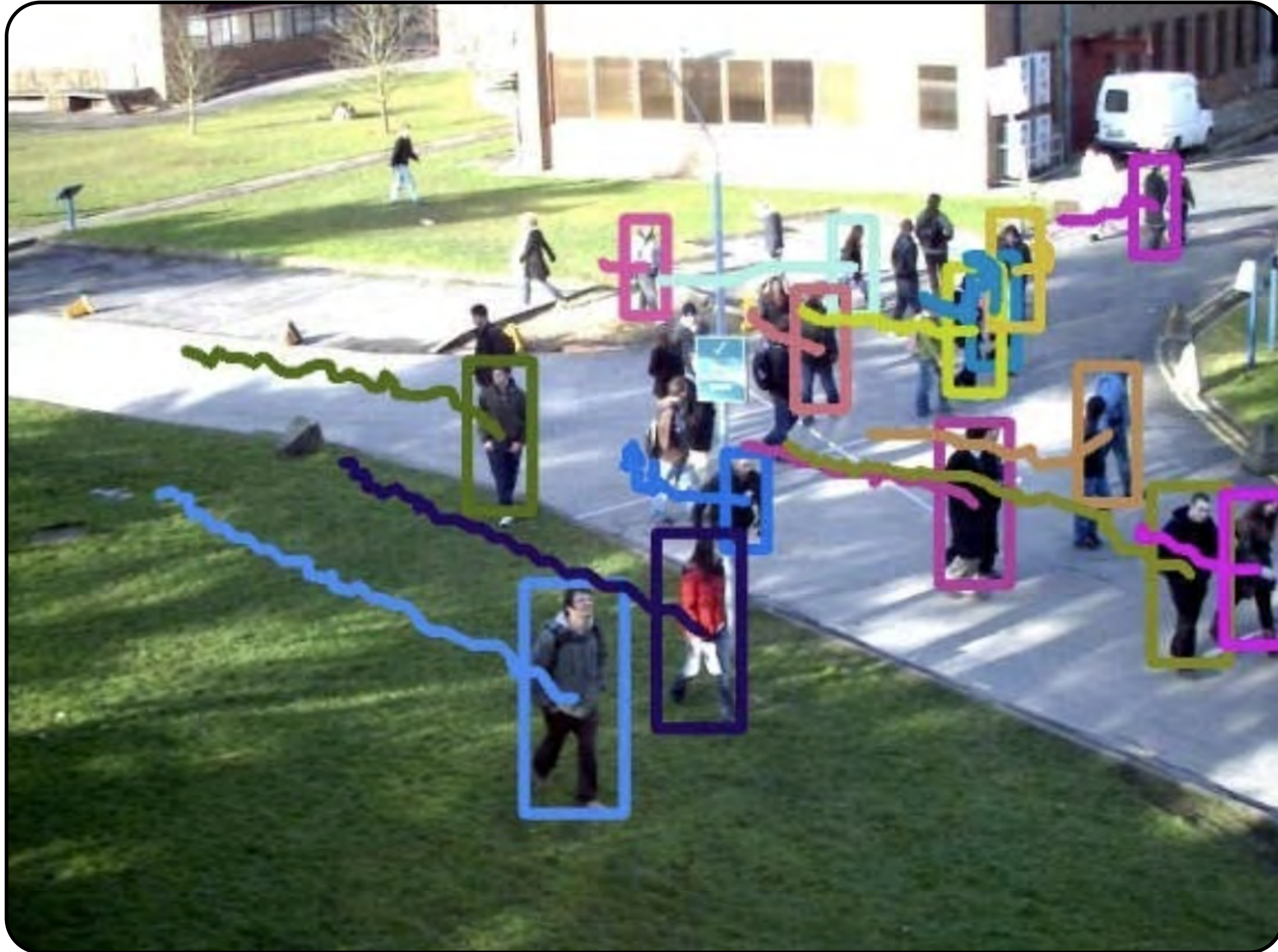
Медленно
обучается, т. к.
на каждом шаге
необходимо
перебрать
большое число
вариантов
простых
классификаторов

Трекинг объектов на видео



4

Трекинг



Постановка задачи

1

Отследить
перемещения
объекта в кадре

2

Предсказать
направления
перемещения

3

Сообщить, если
объект потерян

Сложности

- Несколько объектов на кадре
- Частичное или полное перекрытие другими объектами
- Реидентификация после временной потери из зоны видимости
- Объект может изменять внешний вид во время наблюдения: повороты, масштаб

Детекция и трекинг

- Как правило, детекция сложнее, чем трекинг
- Трекинг использует информацию из предыдущих кадров
- Трекер может накапливать ошибку
- Детектор ищет объект на изображении без учёта предыдущих кадров
- На практике алгоритм детекции запускается реже, чем трекинг

Обзор подходов

Оптический поток (Kanade-Lucas-Tomashi)

- Трекинг характерных точек между кадрами

Калмановский фильтр

- Использует априорную информацию о возможных перемещениях объекта, чтобы предсказывать позицию в будущем

Обзор подходов

Трекинг отдельного объекта

- Детектирует объект на первом кадре, затем отслеживает его перемещения с помощью трекера

Трекинг нескольких объектов

- Детектор для каждого кадра, который выдаёт предсказания. Задача трекера — поставить в соответствие детекции между кадрами

Алгоритмы трекинга в OpenCV

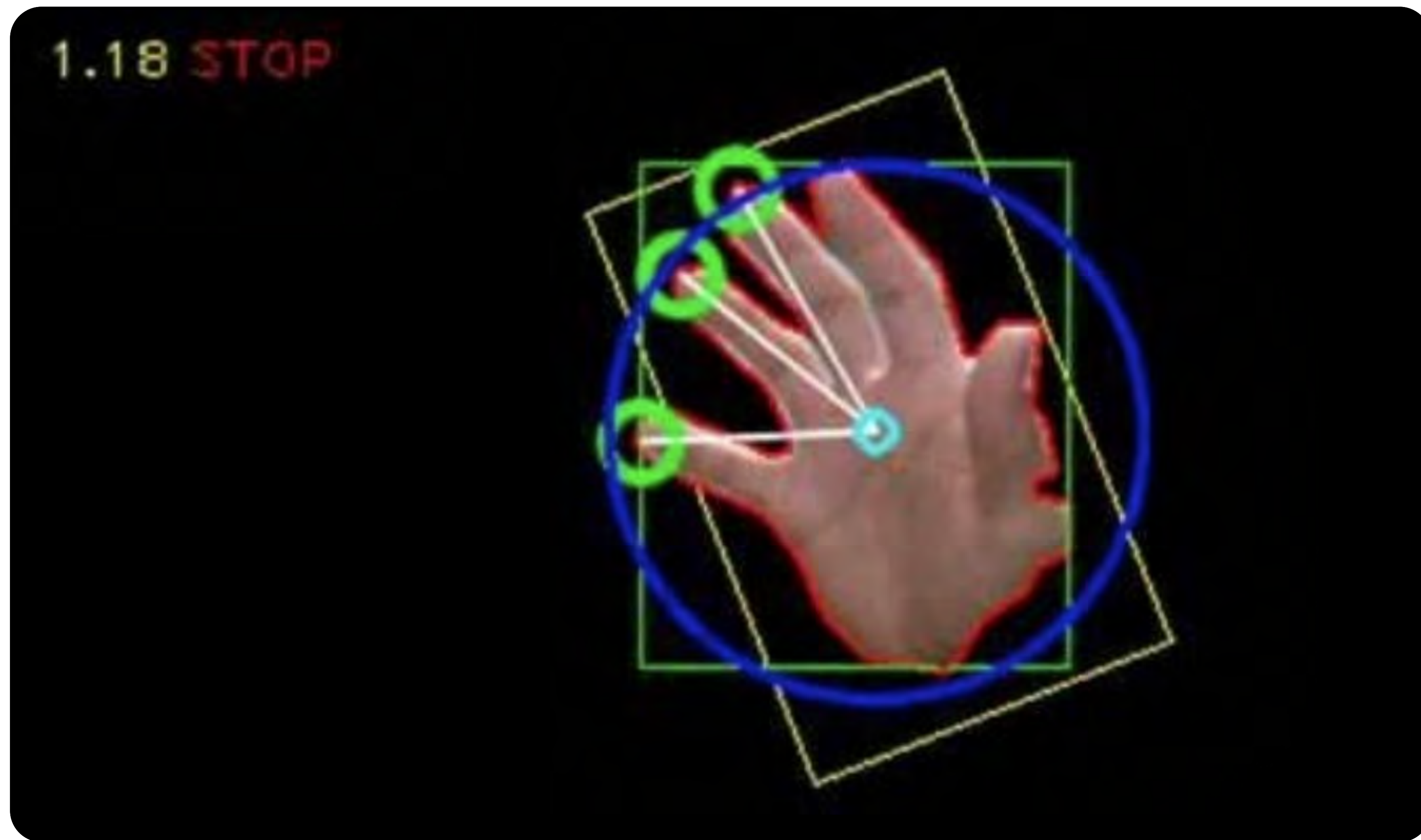
- Для использования трекеров в openCV необходимо установить пакет `opencv-contrib-python`
- Трекеру необходимо хранить состояние между вызовами
- Состояние хранится в объекте трекера
- Создать объект трекера можно с помощью функции `cv2.Tracker<алгоритм_трекинга>_create()`

Пример: распознавание жестов



5

Распознавание жестов



Постановка задачи

Разработать программируемую систему управления компьютером с помощью жестов.

- Жесты распознаются в реальном виде
- Видеопоток поступает в реальном времени через веб-камеру

Этапы решения

1

**Сегментация:
отделение кисти
от фона**

2

**Детекция:
определяем
положение кисти
на изображении**

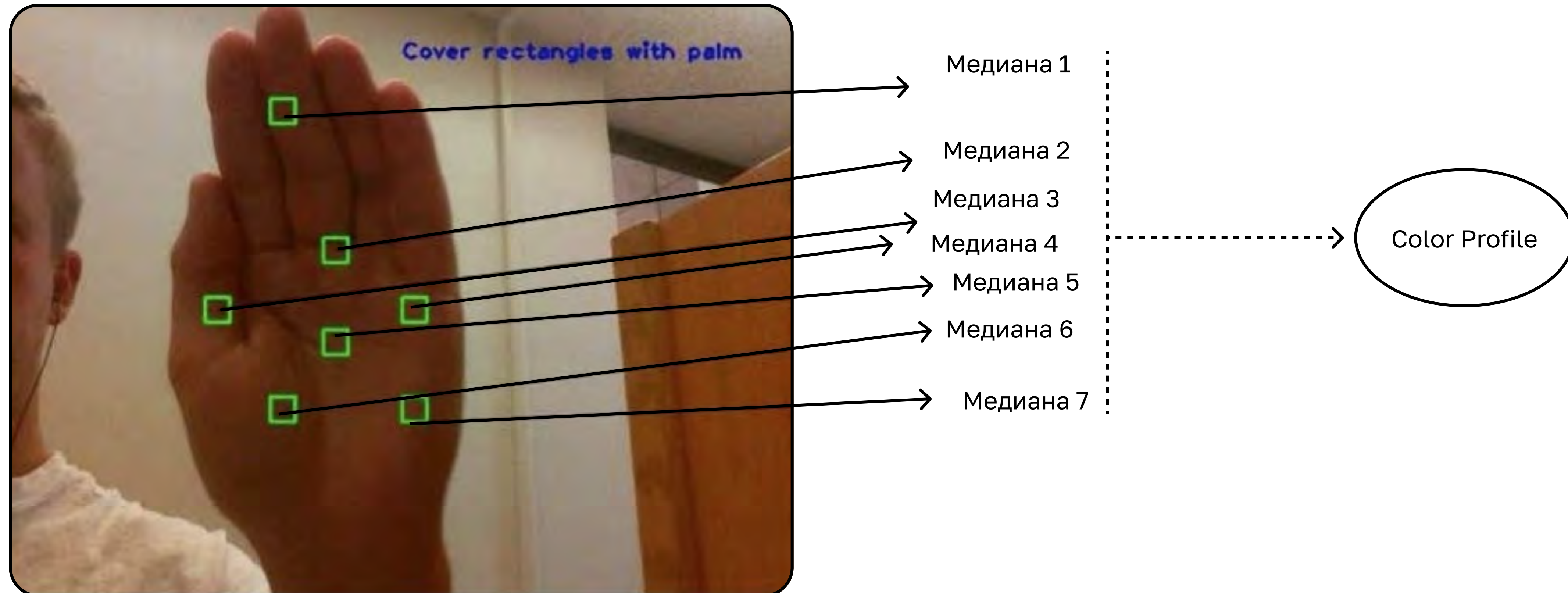
3

**Распознавание
жеста**

Сегментация кисти

- 1 Задаём области на изображении, которые относятся к кисти
- 2 Оцениваем гистограмму цветов в пространстве HSV
- 3 Используем эту оценку для сегментации
- 4 Сегментируем по гистограмме кисть функцией `cv2.calcBackProject`

Сегментация кисти: обратная проекция



Сегментация кисти: обратная проекция



Детекция кисти

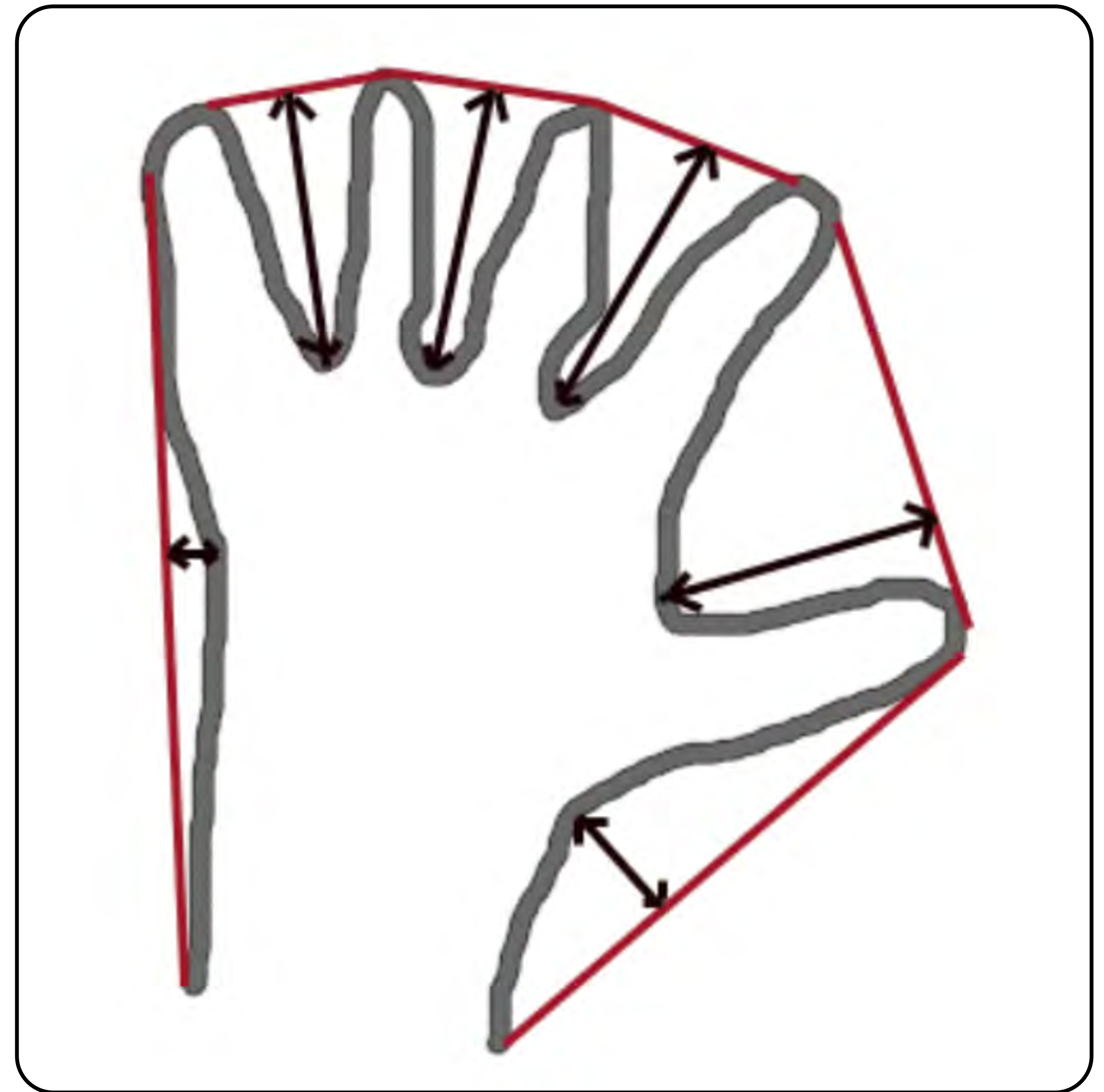
- 1 Находим все контуры сегментированного изображения
- 2 Контур наибольшей площади считаем кистью
- 3 Находим выпуклую оболочку контура с помощью функции `cv2.convexHull`

Распознавание жестов: детекция кисти



Распознавание жестов: детекция кисти

- Оцениваем радиус кисти
- Оцениваем центр ладони
- Удаляем точки на выпуклой оболочке таким образом, чтобы осталось не более пяти точек (одна точка на палец)



Распознавание жестов: детекция КИСТИ

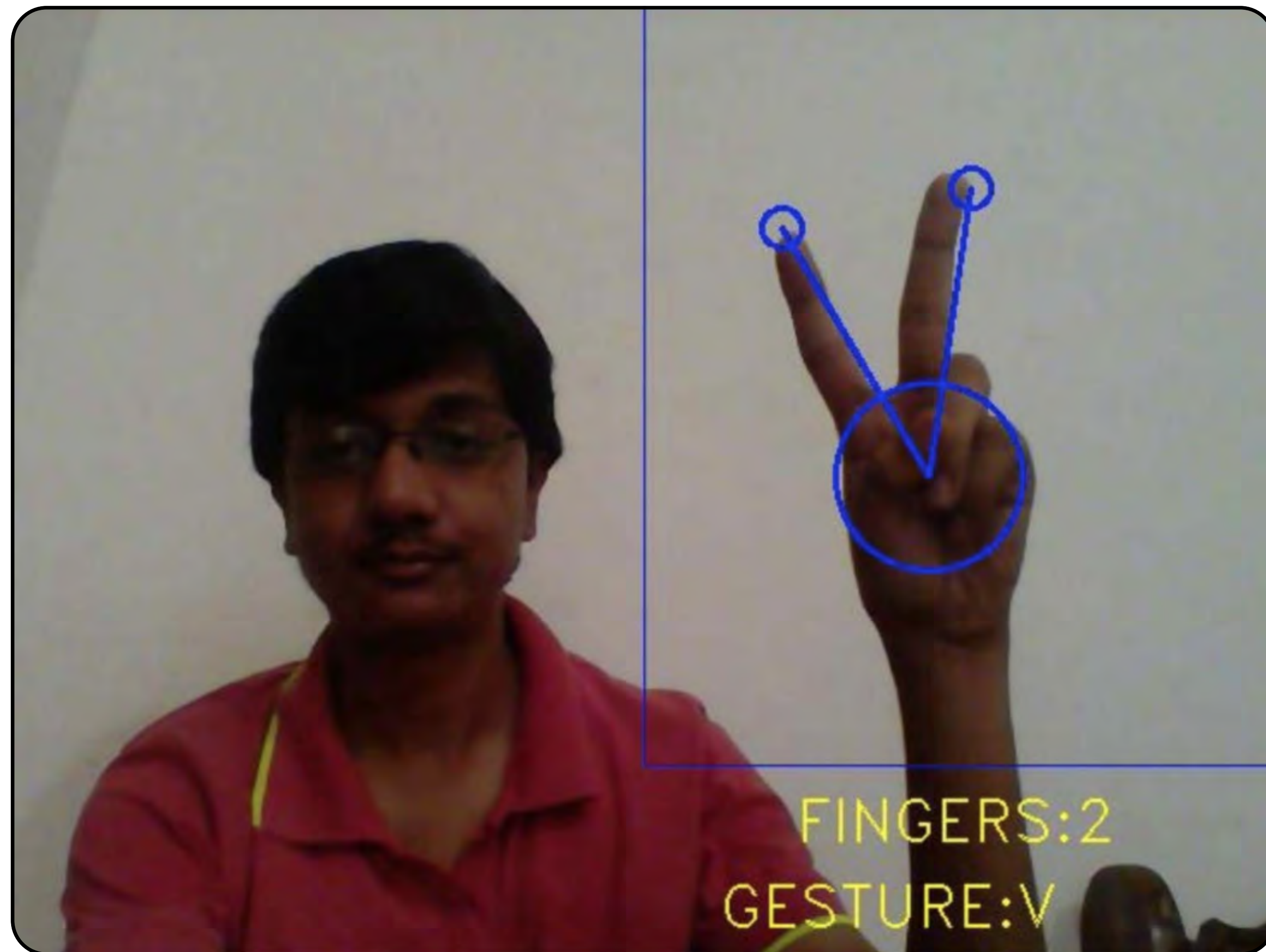


Распознавание жеста

Входные данные: координаты центра ладони, радиус и координаты пальцев.

- Задаём базу размеченных жестов в таком же формате
- Ищем в базе наиболее близкий жест к детектированному

Распознавание жеста



Практика



Практика

Цель: понять базовые принципы работы трекеров и сегментаторов, разобраться, когда необходимо их внедрять.

Задача: рассмотреть работающий трекер лиц, разработать систему сегментации объектов методом водораздела.

Результат:

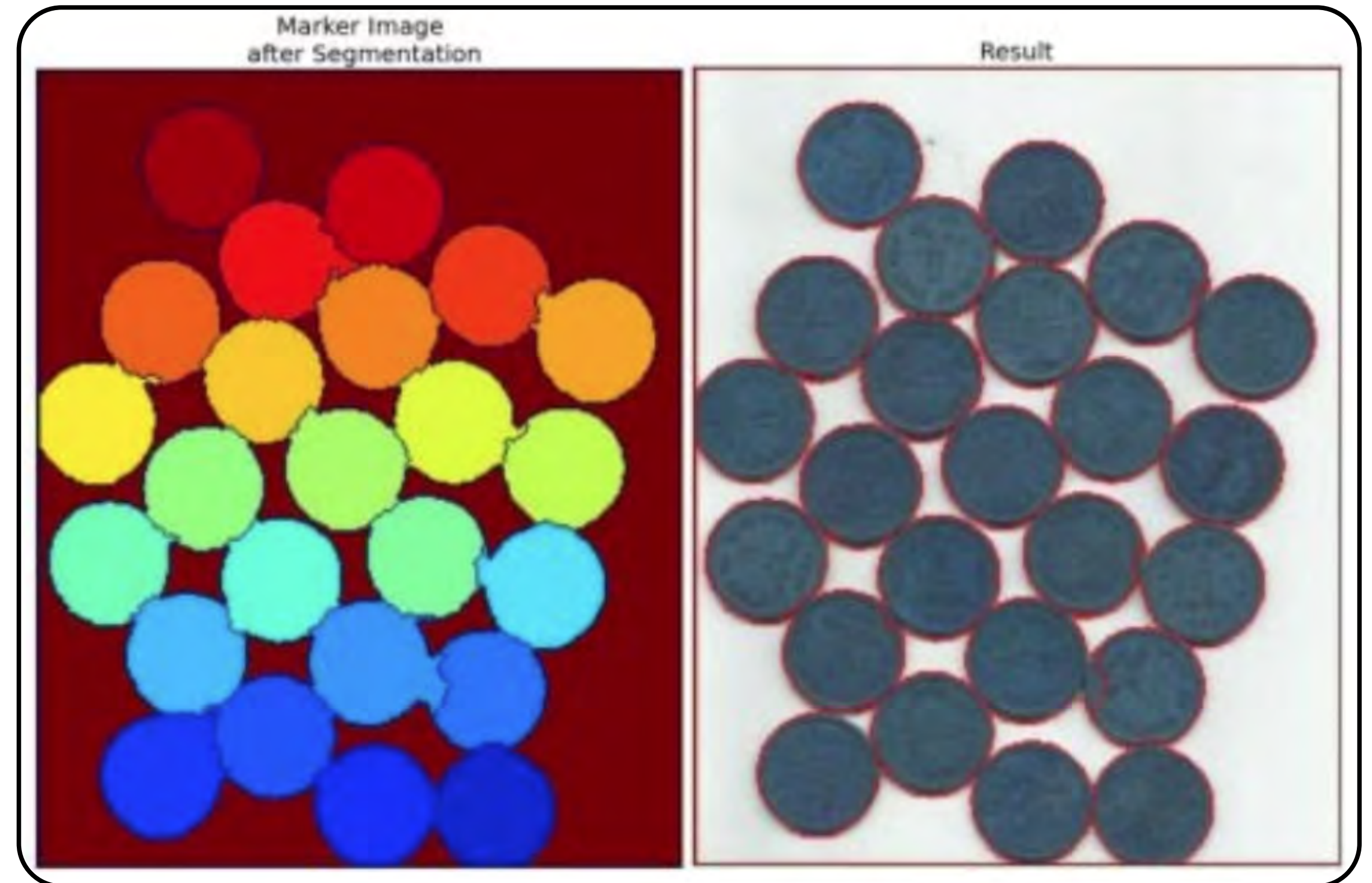
- умение пользоваться предобученным детектором лиц и трекером
- умение проводить сегментацию изображений несколькими способами

Инструменты: Python, OpenCV

Практика

Требуется отделить монеты на рисунке друг от друга.
Сегментация водоразделом:

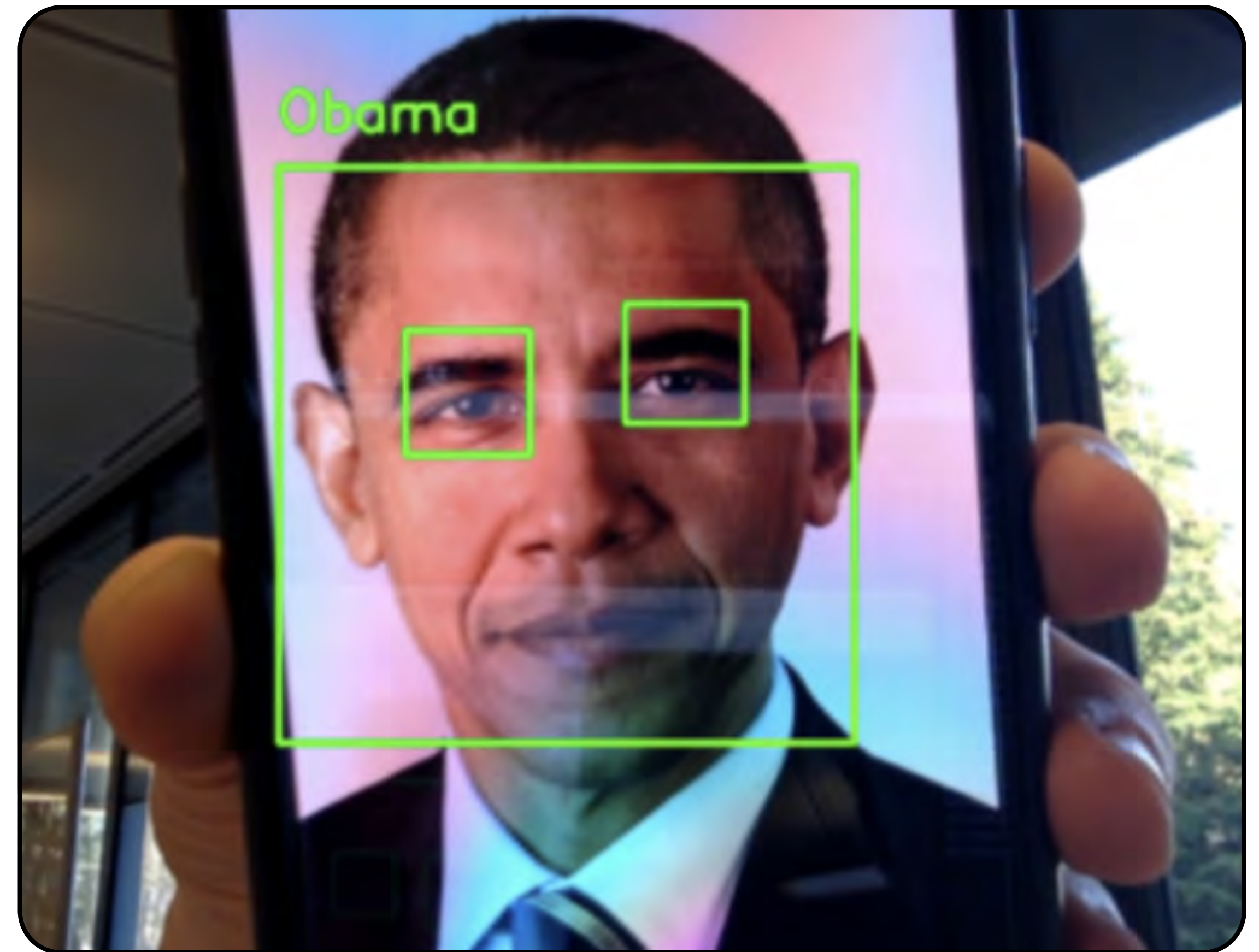
- строим бинарное чёрно-белое изображение
- делаем дистантное преобразование
- ищем центры полученных объектов



Практика

Знакомство с детектором лиц:

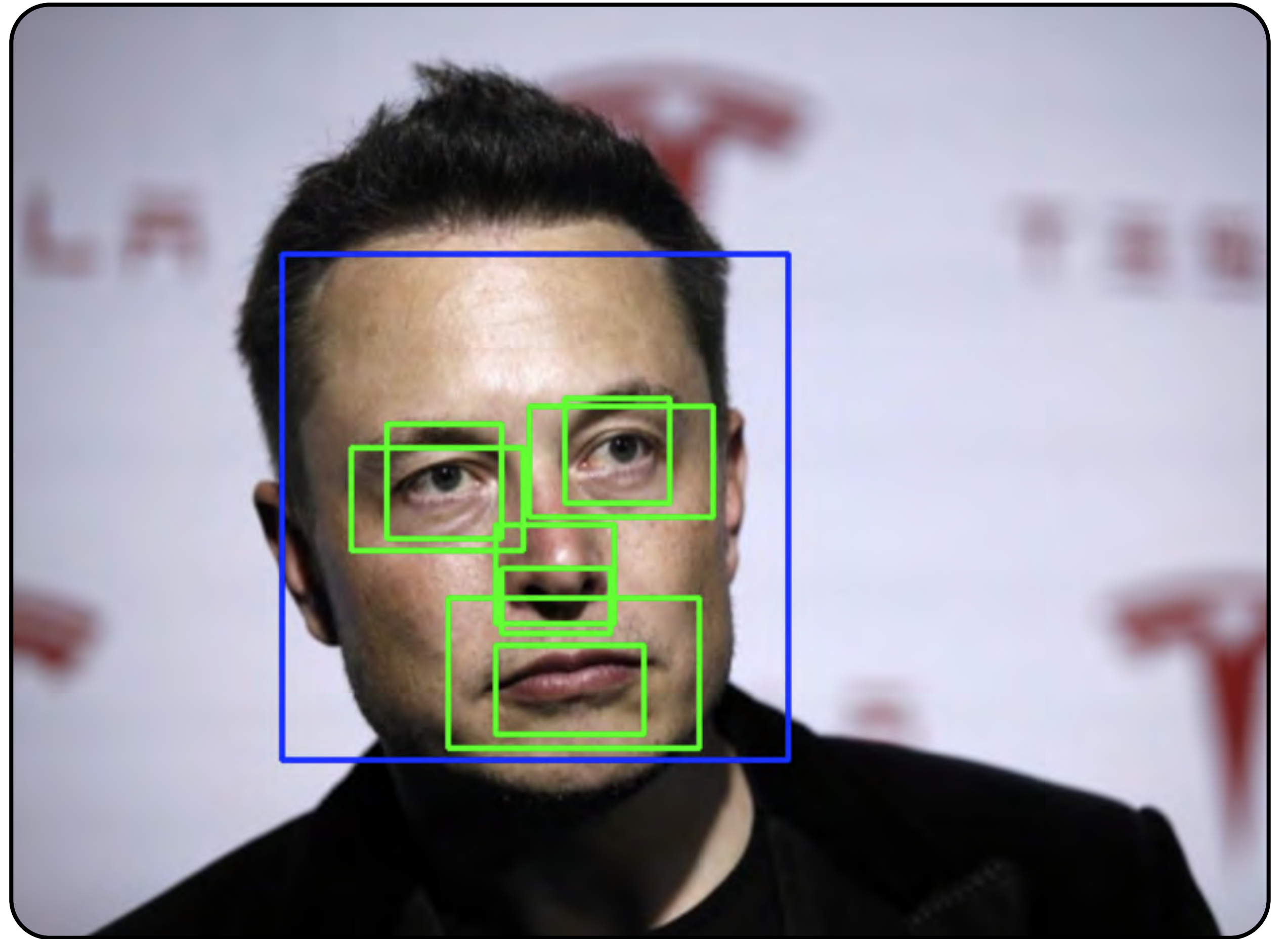
- загружаем предобученный детектор лиц
- визуализируем результаты



Практика

Строим собственный трекер:

- загружаем предобученный детектор лиц
- инициализируем трекер
- анализируем видеофайл

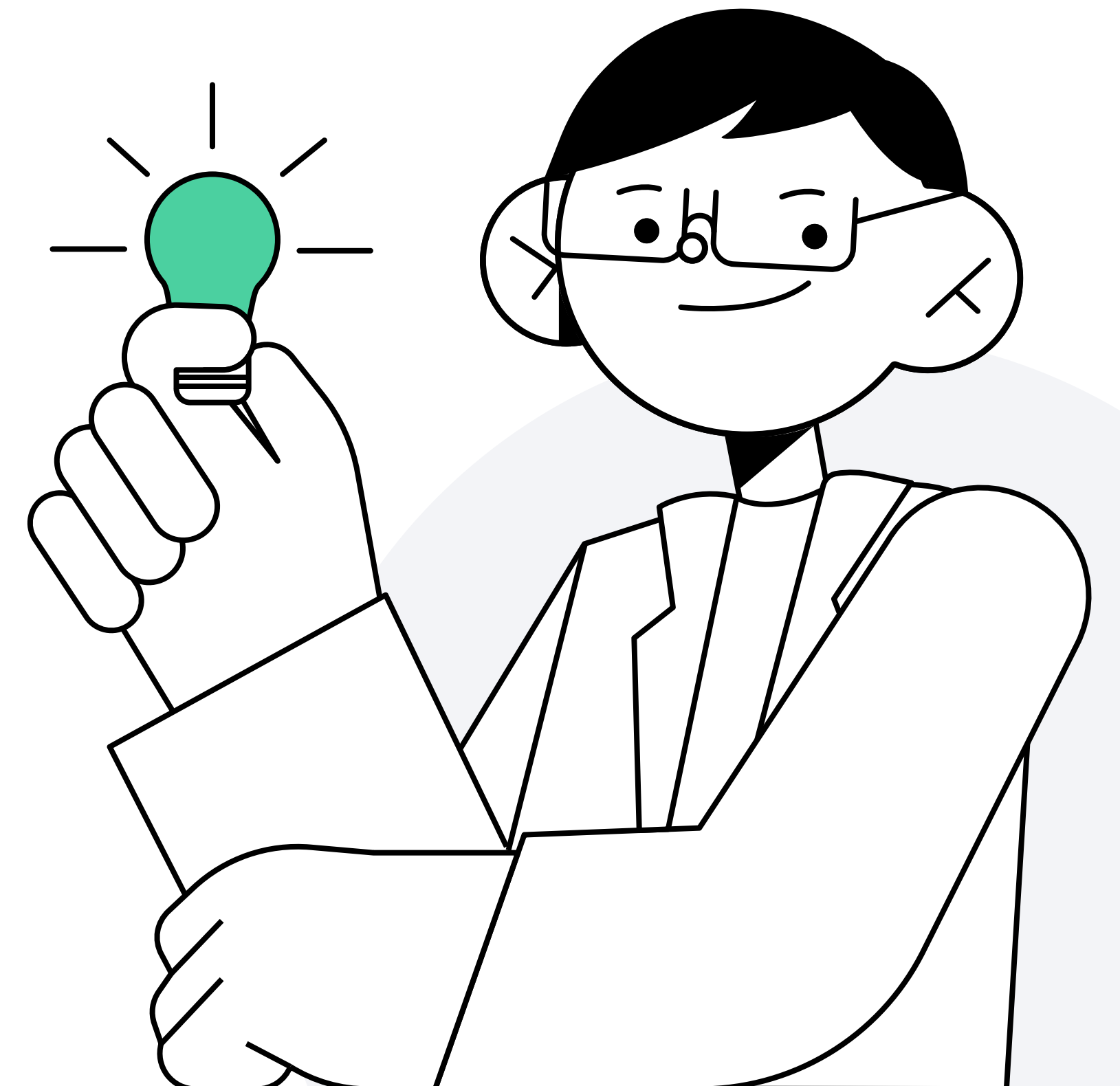


Итоги



Итоги

- 1 Способы сегментации: по порогу, выделение контура объекта, водораздел, суперпиксели
- 2 Каскад Viola-Jones — эффективный способ детекции объектов. В качестве признаков для базовых классификаторов используются разностные свёртки. Метод быстро работает на этапе применения, но требует много времени и большого числа примеров для обучения



Домашнее задание



Домашнее задание

Задание: распознавание рукописного ввода на примере базы MNIST.

Построить классификатор изображений рукописного ввода на базе MNIST.

- В качестве шаблона можно использовать IPython Notebook (002-digit.ipynb)
- Классификатор предлагается строить на признаках, полученных в результате предобработки изображений, например, гистограммы градиентов (HOG) или результат PCA-преобразования
- В качестве модели классификатора можно использовать любую известную вам модель, за исключением свёрточных нейронных сетей

Результат:

- решение необходимо предоставить в IPython Notebook с реализацией процесса построения модели и скриншота с финальным результатом на Kaggle
- чтобы получить зачёт, значение метрики accuracy должно быть больше 0,6. Метрика оценивается на тестовой выборке в пределах контекста [Digit Recognizer] на Kaggle

Инструменты:

- IPython Notebook (002-digit.ipynb)
- тестовая выборка (Digit Recognizer)

Дополнительные материалы



Дополнительные материалы к занятию

- ① Computer Vision: Algorithms and Applications ([Chapter 5](#))
- ② Computer Vision: Algorithms and Applications ([Chapter 14](#))
- ③ Image segmentation ([Wikipedia](#))
- ④ [SLIC](#) Superpixels
- ⑤ Document Image Binarization Techniques, Developments and Related Issues: [A Review](#)
- ⑥ Visual Tracking: [An Experimental Survey](#)

Спасибо за внимание!

Егор Конягин

Сотрудник лаборатории высокопроизводительных вычислений и больших данных в Сколтехе

